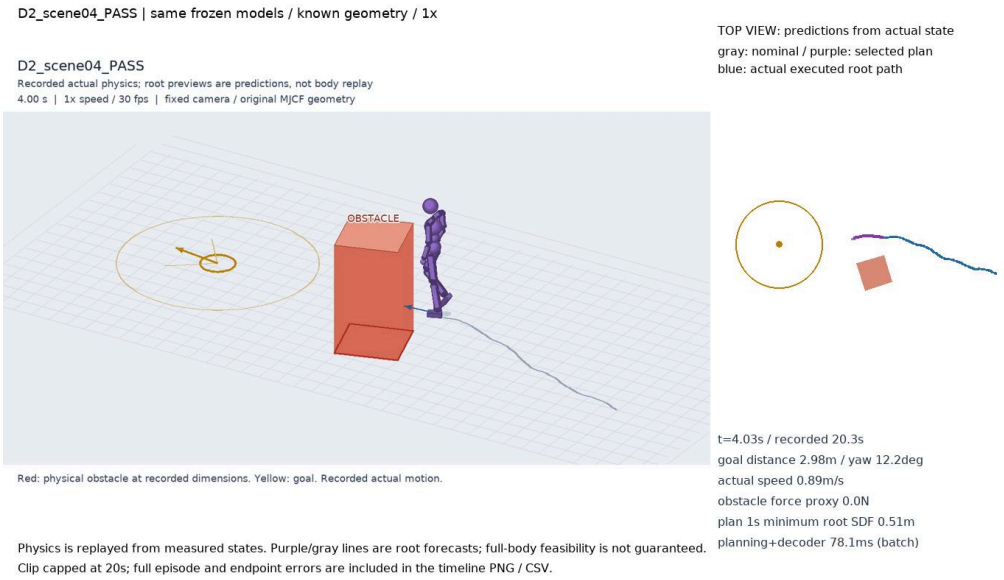
INTRO

비용 유도 휴머노이드 모션 생성

Cost-Guided Humanoid Motion Generation for Test-Time Obstacle Adaptation

개인 Robot Learning 프로젝트 · 최종 설명 보고서



학습된 움직임을 장애물 비용으로 바꾸고 실제 물리 시뮬레이션에서 실행한 장면.

배운 움직임을 다시 학습하지 않고, **새 장애물 조건에 맞춰 바꿀 수 있을까?**

기본 모션 준비에서 PPO·AMP teacher, 실제 데이터 수집, MLP·Diffusion 학습, 공통 실행기, 비용 유도, 물리 평가와 실패 진단까지 설명한다.

기준일 2026년 11월 5일 · 최신 실험 Guidance Comparison V2
대상 단일 generic humanoid / 평지 / 알려진 정적 장애물 하나
읽는 목적 배경지식부터 구현 판단과 결과의 한계까지 스스로 설명하기

CONTENTS

전체 흐름과 빠른 찾기

처음에는 앞에서부터 읽고, 발표 준비 때는 구조·결과·한계로 돌아온다.

문제와 물리 제어의 기초에서 시작해 RL·데이터·생성·실행으로 이어진다. 결과는 성공률과 실패 원인을 함께 읽고, 재현과 근거 파일은 뒤의 부록에서 확인한다.

직접 찾아가는 주요 장

무엇을 만들고 검증했나 05
학습할 때와 실행할 때 06
PPO는 teacher를 어떻게 개선하나 09
DeepMimic과 AMP의 차이 10
Teacher는 어떻게 걷고 방향을 맞추나 12
한 번의 rollout이 학습 자료가 되는 과정 14
660개 입력에서 60×99개 미래 값으로 18
MLP는 무엇을 학습했나 19
Diffusion은 왜 노이즈에서 시작하나 20
왜 PPO tracker에서 decoder로 갔나 22
Decoder는 어떻게 관절 목표를 만드나 23
무엇을 나쁜 미래라고 판단했나 26
비용이 Diffusion을 어떻게 바꾸나 27
0/24 실패를 그대로 결론으로 쓰지 않았다 29
같게 맞춘 것과 다른 것 30
무엇을 몇 번 평가했나 31
어느 정도면 도착해 멈췄다고 하나 32
동일 비용 아래에서 무엇이 달랐나 34
작은 장애물에서는 강하고 벽에서는 약하다 35
실패를 생성과 실행으로 어떻게 나눴나 37
같은 원통, 유도 전후의 차이 38
같은 상자, MLP와 Diffusion 39
벽은 왜 계속 밀고 있었나 41
무엇을 기여라고 설명할 수 있나 43
무엇이 아직 미완료인가 44
최신 모델과 파일을 찾는 법 45
기존 로컬 환경에서 다시 평가하기 46
어떤 연구를 참고했나 50
최종 설명의 근거를 추적하는 법 51

INTRO

이 보고서를 읽는 순서

개념을 익힌 뒤, 데이터가 어떻게 행동으로 바뀌는지 따라간다.

이 문서는 예상 질문 목록이 아니라 시스템 자체를 이해하기 위한 설명서다. 처음 읽을 때는 수식의 모든 기호를 외우기보다, 각 단계의 **입력·출력·학습 정답·실행 역할**을 구분하는 것이 중요하다.

읽을 범위	핵심 내용	이해할 연결
1부 · 문제와 배경	연구 질문, 물리 제어, RL·PPO·AMP	왜 모션 파일만으로 로봇을 걷게 할 수 없는가
2부 · 데이터와 모델	실제 rollout, 좌표·정규화, MLP·Diffusion	어떤 경험을 어떤 정답으로 학습했는가
3부 · 실행과 유도	decoder, 재계획, 장애물 비용, MLP 제약	생성한 숫자가 어떻게 실제 행동을 바꾸는가
4부 · 결과와 판단	평가 설계, 성공률, 영상, 실패 원인	어디까지 검증했고 무엇은 아직 모르는가
부록 · 재현과 근거	파일 계보, 명령, 용어, 출처	설명에 필요한 근거를 어디서 찾는가

문서의 증거 수준

사실은 최종 코드·설정·checkpoint·로그에서 확인한 내용이다. **해석**은 관찰에서 가능한 설명이며 원인 확정과 구분한다. **후속**은 앞으로 할 수 있는 작업으로, 실행 완료처럼 쓰지 않는다.

> **숫자를 읽는 법**
>
> 23/24는 24번 평가 중 23번 성공했다는 뜻이다. 서로 다른 장면 수, 학습 seed, sampling seed, 평가 횟수는 같은 수가 아니다. 95.8%는 시험한 제한된 장면에서의 비율이며 모든 상황의 성공 확률을 보장하지 않는다.

INTRO

최종 결론부터 이해하기

작은 장애물에서 작동 가능성을 확인했고, 넓은 장애물과 실행 안정성의 한계도 확인했다.

최신 동일 조건 비교	작은 원형·상자	전체 장면
MLP 원본	7/12 · 58.3%	8/24 · 33.3%
제약 MLP 출력 최적화	6/12 · 50.0%	9/24 · 37.5%
Diffusion 원본	0/24 · 0%	0/48 · 0%
경로 힌트 없는 Diffusion 비용 유도	23/24 · 95.8%	33/48 · 68.8%
별도 기준선: analytic root 보정	12/12 · 100%	24/24 · 100%

주 결과는 **생성 후 root 경로 수정이나 우회 곡선 지시 없이**, Diffusion 생성 도중 비용을 적용했을 때 작은 장애물의 무충돌 도착·정지가 가능해졌다는 것이다. 생성기와 decoder의 가중치는 이 장애물 평가 동안 추가 학습하지 않았다.

동시에 큰 물체·벽에서는 유도 방식이 10/24에 그쳤다. Root 보정은 강한 별도 기준선이었고, MLP 최적화는 속도 폭주를 줄여도 낙상이 남았다. 따라서 “Diffusion만이 풀 수 있는 문제” 또는 “완벽한 navigation”으로 결론내리지 않는다.

> **핵심 해석**
>
> 비용 유도는 원본 Diffusion의 실패를 크게 줄였다. MLP와 같은 정보·비용·실행기를 사용한 비교에서도 이번 고정 checkpoint는 더 높은 점수를 냈다. 하지만 모델 구조·학습 목적·최적화 방식·계산 예산까지 같지는 않으므로 Diffusion 구조 자체의 보편적 우월성은 입증하지 않았다.

1부 · 문제 정의

무엇을 만들고 검증했나

“어떻게든 걷는가”에서 “학습된 움직임을 조건에 맞게 바꿀 수 있는가”로 질문을 좁혔다.

입력과 원하는 결과

현재 휴머노이드의 상태와 최근 움직임, 목표 위치와 도착 방향을 준다. 시뮬레이터가 알고 있는 장애물 위치·크기를 비용 계산에 사용한다. 원하는 결과는 넘어지거나 장애물에 닿지 않고 목표 허용 범위에 도착해 방향을 맞추고 낮은 속도로 서 있는 것이다.

왜 의미가 있는가

정책이 한 과제를 잘하더라도 환경이 바뀔 때마다 보상과 데이터 수집, 학습을 다시 해야 한다면 적용 비용이 커진다. 이미 학습한 움직임을 유지하면서 실행 시 목적을 바꿀 수 있는지는 로봇 학습의 중요한 실용 문제다. 여기서는 그 전체 문제 중 “알려진 정적 장애물 하나”라는 작은 범위를 골랐다.

연구 질문을 두 층으로 나눈다

- 주 질문: 학습된 모션 분포를 실행 시 장애물 비용으로 유도해, 경로 후처리 없이 실제 무충돌 도착·정지에 기여할 수 있는가?
- 비교 질문: 같은 입력 정보와 비용·실행기에서 MLP 출력 최적화와 Diffusion 생성 중 유도는 성공률·낙상·계산 시간에 어떤 차이를 보이는가?

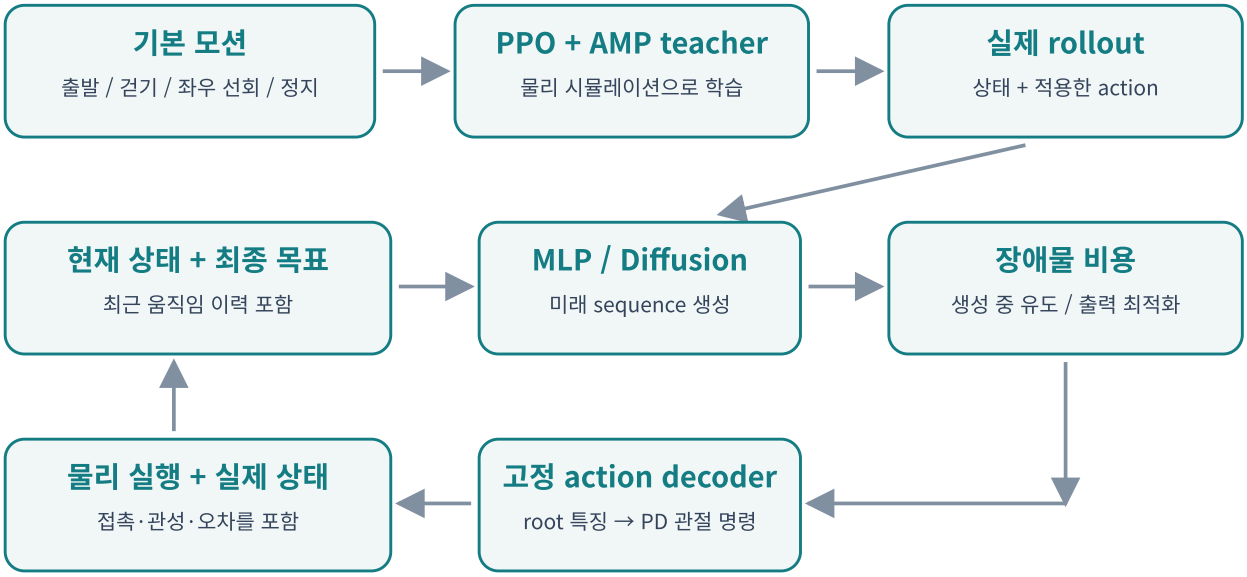
범위

Generic humanoid 한 종류, 평지, 단일 정적 장애물, 알려진 geometry를 사용한다. 카메라 인식·SLAM·동적 장애물·복잡 지형·실제 로봇은 포함하지 않는다. 최신 평가는 이미 걷고 있는 실제 상태에서 시작하며, 정지 출발부터 모든 방향을 가는 독립적인 전체 navigation 평가가 아니다.

1부 · 전체 구조

학습할 때와 실행할 때

Teacher는 데이터를 만들고, 생성기는 미래를 제안하며, decoder는 실제 관절을 제어한다.



위쪽은 경험을 모으고 모델의 가중치를 바꾸는 **학습 단계**다. 아래쪽은 이미 학습한 가중치를 고정한 채 새 입력에 반응하는 **실행 단계**다. 장애물 비용을 추가하는 것은 실행 중 매번 PPO를 학습한다는 뜻이 아니다.

역할	무엇을 받나	무엇을 내나
Teacher	상태·목표	수집용 실제 관절 action
생성기	최근 상태·action 이력·목표	미래 state-action sequence
실행 decoder	실제 현재 관측·미래 root 특징	실제 적용할 관절 PD 목표

1부 · 물리 제어의 기초

모션과 action은 다른 것이다

화면에서 자세를 재생하는 것과, 힘으로 그 자세에 도달하는 것은 다르다.

모션 파일은 “어떤 자세였는가”를 기록한다

모션에는 시간에 따른 몸통 위치와 회전, 관절 자세 등이 들어 있다. 화면의 캐릭터 pose를 이 값으로 매 순간 강제 설정하면 물리적으로 불가능한 자세도 그럴듯하게 보일 수 있다. 발이 지면에 닿는 동안 미끄러지거나 몸통이 순간 이동해도, 단순 재생만으로는 드러나지 않을 수 있다.

물리 제어는 “어떻게 움직일 것인가”를 실행한다

실제 물리 시뮬레이션에서는 현재 속도와 관성, 중력, 발 접촉, 관절 제한이 다음 상태를 결정한다. Action이 목표 관절 자세를 제시하더라도, 몸이 즉시 그 자세로 바뀌는 것은 아니다. 목표를 향한 관절 구동을 물리 엔진이 적분해 결과를 만든다.

PD 제어의 개념: torque ≈ Kp × (q_target − q_actual) − Kd × dq_actual

q는 관절 각도, dq는 각속도다. Kp는 목표 자세로 끌어당기는 힘의 크기, Kd는 속도를 감쇠시키는 정도를 뜻한다. 위 식은 제어 원리를 설명하는 근사이며 backend의 실제 drive·force limit·구현 세부는 다를 수 있다.

이 프로젝트의 실제 의미

Action은 28개 자유도에 대한 position drive 목표다. 미래 state71이나 한 모션 frame의 길이를 action 차원으로 착각하면 안 된다. Root 위치·방향은 시뮬레이터의 관절 action으로 간접 제어되며, 평가 도중 목표 root 위치로 강제 옮겨 성공시키지 않는다.

> **정지에서 자주 생긴 문제**
>
> 이미 앞으로 움직이는 몸에 고정 자세만 갑자기 요구하면 운동량이 남아 몸이 계속 나아간다. 발 위치가 받쳐 주지 못하면 앞이나 뒤로 넘어질 수 있다. 정지는 속도가 0인 그림 한 장이 아니라, 감속·발 디딤·균형 유지가 연결된 과정이다.

1부 · RL 기초

강화학습은 어떤 데이터를 배우나

행동의 정답을 매 순간 주지 않고, 행동의 결과를 보상으로 평가한다.

한 번의 transition

(관측 o_t, 행동 a_t, 보상 r_t, 다음 관측 o_(t+1), 종료 여부 done)

로봇이 현재 몸 상태를 관측하고 action을 낸다. 물리 엔진이 한 제어 간격을 진행하면 다음 상태가 생긴다. 환경은 목표에 가까워졌는지, 원하는 방향으로 움직였는지 등을 보상으로 계산한다. 이 한 묶음이 transition이며, 여러 묶음이 한 episode를 만든다.

Policy와 value

Policy는 관측을 받아 행동을 고르는 함수다. 신경망으로 표현된 policy는 가중치가 바뀌면 행동도 달라진다. **Value**는 현재 상태에서 앞으로 받을 보상의 예상값이다. 실제 경험이 예상보다 좋았는지 나빴는지를 비교해 행동을 조정할 수 있다.

Return과 성공률은 다르다

Return은 보통 episode 안에서 받은 보상의 합 또는 할인된 합이다. 오래 버티기만 해도 합이 커질 수 있고, 움직임의 스타일 보상이 높아도 목표를 놓칠 수 있다. 따라서 학습 return 상승만으로 원하는 task를 잘한다고 판단하지 않았다. 실제 위치·방향·속도·충돌·낙상·정지 유지로 별도 평가했다.

Iteration, sample, episode, epoch

용어	의미
sample	환경에서 얻은 transition 한 개
episode	reset부터 종료까지의 한 실행
PPO iteration	일정 rollout을 모은 뒤 policy/value 등을 갱신하는 한 주기
gradient update	배치 하나로 가중치를 갱신한 한 번
epoch	학습 자료 전체를 한 번 순회하는 의미. PPO 설정·배치 표기와 구분

최신 H60 생성기 8,000 updates는 teacher의 PPO 8,000 iteration이라는 뜻이 아니다. 다른 모델·다른 데이터·다른 최적화 단위다.

1부 · PPO

PPO는 teacher를 어떻게 개선하나

경험 수집과 가중치 갱신을 번갈아 수행한다.

실제 반복 흐름

- 현재 actor(policy)로 여러 환경에서 action을 샘플링하고 transition을 모은다.
- Critic(value network)으로 미래 보상을 추정하고 advantage를 계산한다.
- 좋았던 행동의 확률은 높이고 나빴던 행동의 확률은 낮추도록 actor를 갱신한다.
- 새 정책이 이전 정책에서 과도하게 벗어나지 않도록 clipped objective를 사용한다.

$$\text{ratio} = \pi_{new}(a|o) / \pi_{old}(a|o)$$
$$L_{clip} = \text{mean}[\ \min(\text{ratio} \times A,\ \text{clip}(\text{ratio}, 1-\varepsilon, 1+\varepsilon) \times A)\ ]$$

A는 advantage로, 그 행동이 예상보다 얼마나 좋았는지를 나타낸다. ε는 업데이트 변화를 완화하는 값이며 해당 goal agent 설정의 clip ratio는 0.2다. 이 clipping이 행동의 물리적 안전을 보장하는 것은 아니다.

이 프로젝트의 구현 위치

PPO의 actor·critic·학습 루프와 AMP discriminator는 외부 MimicKit 구현을 사용했다. 프로젝트에서는 목표 조건, 보상, 데이터 수집과 평가를 구성했다. “PPO 알고리즘을 처음부터 새로 만들었다”는 표현은 정확하지 않다.

확인한 goal agent 설정	값
Actor / critic / discriminator	각 fc_2layers_1024units
할인율 γ / TD λ	0.99 / 0.95
rollout 길이	steps_per_iter = 32
actor / critic epoch	5 / 2
task / discriminator reward 비중	0.65 / 0.35

이 값은 보존된 agent_sequential_v31.yaml 기준이다. 긴 개발 기간 전체가 한 번의 동일 설정 학습이었던 것은 아니며, checkpoint 계보와 추가 학습 기록을 구분해야 한다. [1]

1부 · 모션 사전지식

DeepMimic과 AMP의 차이

모범 자세를 시각별로 따라갈 것인가, 모션다운 움직임의 분포를 배울 것인가.

DeepMimic: 시간에 맞춘 모션 추종

DeepMimic은 reference와 실제 캐릭터의 자세·속도·몸통·주요 부위 위치 차이를 이용한 모방 목표를 RL에 결합한다. 초기 reference tracker는 외부 reference target을 관측하고 따라가도록 학습했다. 모션 파일을 화면에서 재생하는 것과 달리, 관절 구동과 물리를 거쳐 자세를 실현해야 한다. [2]

AMP: 움직임의 스타일을 보상으로

AMP는 실제 시뮬레이션 움직임과 reference 모션의 짧은 관측 이력을 discriminator가 구분하도록 학습한다. Policy는 task 보상과 함께 "reference 데이터의 움직임처럼 보이는가"라는 스타일 보상을 받는다. 모든 순간이 모션의 동일 시각과 일치해야 하는 강한 phase 동기화를 줄일 수 있다. [3]

$$r_{train} = 0.65 \times r_{task} + 0.35 \times r_{AMP}$$

해당 MimicKit 구현: $r_{AMP} = 2 \times [-\log(\max(1 - D(obs_history), 0.0001))]$

여기서 D는 reference 같은 움직임이라고 판단하는 확률이다. AMP reward는 agent에서 더해지므로 환경의 task reward 안에 중복해서 더하지 않는다. Discriminator가 목표 도착까지 판단하는 것은 아니므로 task reward가 별도로 필요하다.

> **현 teacher는 단순한 "보상 하나 추가"가 아니다**
>
> Goal teacher에는 AMP 외에도 목표 진행·전진·회전·정지·넘어짐 종료, 상체와 모션 품질·미끄러짐 등의 항목이 있다. phase-free 가까운 전체 자세/속도 reference를 활용한 품질 보완도 포함한다. 따라서 순수 AMP 하나만으로 모든 동작을 얻었다고 설명하면 안 된다.

1부 · 기본 데이터

어떤 모션에서 출발했나

소수의 동작과 그 파생본을 출발점으로 삼았지만, 최종 학습 표본은 실제 rollout이다.

초기 다섯 역할	보존된 초기 파일 예
출발	mixamo_start_xyfix.pkl
걷기	humanoid_walk_neck_fixed.pkl
좌회전	humanoid_turn_left_5s_8s.pkl
우회전	humanoid_turn_right_5s_8s.pkl
정지	mixamo_stop_xyfix.pkl → 후속 stop/hold 파생본

사용자가 준비한 Mixamo 데이터와 MimicKit 기반 모션을 변환·구간 추출·좌우 대칭·정지 유지 보완했다. 초기 five-motion YAML은 다섯 reference의 목록이지, physical rollout dataset이 아니다. 최종 teacher의 모션 prior 목록에는 보완된 stop/hold와 대칭 파생본까지 10개 항목이 있다.

원본과 파생본을 구분해야 하는 이유

같은 walk를 좌우 반전하거나 한 구간만 잘라도 완전히 새로운 독립 mocap을 얻은 것은 아니다. 같은 source group으로 묶어 split 누출을 피해야 한다. “원본 clip 다섯 개”와 “모든 학습 상황 다섯 개”는 다르다. RL은 그 prior로부터 수많은 물리 transition을 생성한다.

리타겟팅에서 확인한 것

원본 skeleton과 generic humanoid의 관절 구조·축·길이가 달라 좌표와 회전을 맞춰야 했다. 발등이 서거나 발이 지면을 뚫는 문제, jitter, 반복 시 root 점프를 점검했다. 화면상 자연스러움 검사는 필요하지만 충분하지 않아, 실제 물리 추종과 정지·접촉을 별도로 확인했다.

> **현재 평가 대상**
>
> 이전 G1 속도 학습은 개발 이력이다. 최종 비교는 28 DoF·약50.05kg의 generic humanoid이며 G1 checkpoint를 같은 모델처럼 사용하지 않는다. Run·실제 로봇·terrain을 성공 범위에 포함하지 않는다.

1부 · TEACHER의 목표

Teacher는 어떻게 걷고 방향을 맞추나

목표로 향하는 방향과 도착해서 바라볼 방향을 구분한다.

관측에 들어가는 목표

World 좌표의 최종 목표 위치와 방향을 보존하고, 현재 몸의 heading 기준 상대 위치와 방향으로 관측을 만든다. 가까워질수록 목적지까지의 오차가 달라지며, 목표가 뒤쪽이면 먼저 몸을 돌려 앞으로 이동하도록 보상과 관측을 구성했다.

이동과 도착 정렬

최종 실행 환경은 이동 중 목표 위치 쪽을 보게 하고, 충분히 가까워져 속도가 낮아지면 최종 방향 정렬을 요청한다. Alignment 진입은 거리 0.24m 이하와 속도 0.18m/s 이하이며, 거리 0.6m 밖으로 나가면 이동 단계로 복귀한다. 이런 서로 다른 진입·이탈 문턱을 hysteresis라고 한다. 경계에서 단계가 매 순간 바뀌는 것을 줄인다.

방향 요청은 한 번에 최대45도로 제한하고, arrival turn_route는 left다. 따라서 도착 방향 처리에는 기존 실행 보조가 남아 있다. 생성 모델이 아무 보조 없이 최단 회전 방향까지 모두 결정한다고 설명하면 안 된다. 최종 목표 자체를 매 재계획마다 새로 만들거나 목표 도착 시각을 무한히 연장하지 않는다.

보상의 큰 역할

역할	필요한 이유
진행·전진	옆으로 끌거나 목표 근처에서 아무것도 안 하는 해법을 줄임
회전·정렬	이동 방향과 최종 바라보는 방향을 단계적으로 맞춤
감속·정지	위치·방향만 맞고 계속 움직이는 상태를 구분
AMP·모션 품질	task 점수만 높이는 부자연스러운 걸음 억제
미끄러짐·후진 억제	발 끌기와 뒤로 가는 보상 편법을 줄임

Teacher 훈련의 위치 허용 범위(예:0.2m)와 최신 비교 평가의 허용 범위(1.25m)는 서로 다른 목적의 값이다. 이 둘을 같은 “정지 조건”으로 섞지 않는다.

1부 · 보상 설계의 실제

보상을 높여도 좋은 행동이 나오지 않는 이유

신경망은 사람이 의도한 의미보다, 주어진 점수를 높이는 방법을 배운다.

목표 속도와 감속

$$\text{remaining} = \max(\text{distance} - \text{stop_deadband}, 0)$$
$$\text{desired_speed} = \min(\text{walking_speed}, \text{deceleration_gain} \times \text{remaining})$$

거리와 무관하게 높은 속도를 요구하면 목표를 지난 뒤에야 멈추기 쉽다. 남은 거리에 따라 목표 속도를 줄이면 접근 단계부터 감속하도록 유도할 수 있다. Steering 보완은 현재 heading과 목표 방위가 크게 다를 때 전진 속도 요구를 낮춰, 옆이나 뒤로 끄는 행동을 줄인다.

정지 보상의 개념

$$r_{stop} = \exp[-(\text{위치 오차} / \sigma p)^2 - (\text{방향 오차} / \sigma yaw)^2 - (\text{이동 속도} / \sigma v)^2 - (\text{회전 속도} / \sigma w)^2]$$

위치만 맞고 빠르게 지나가거나, 방향만 맞고 계속 걸어도 높은 정지 점수를 받지 않도록 여러 항목을 함께 본다. 그러나 지수 보상은 오차가 클 때 gradient 신호가 작아질 수 있어 방향 drive·정렬 점유 등의 항목도 보완했다.

실제로 관찰한 보상 편법

초기에는 짧게 한 걸음만 내딛고 발을 끌거나, 한쪽 보폭만 커지거나, 목표 허용 영역에 들어가려다 상체 자세가 무너지는 문제가 있었다. 움직임 스타일을 너무 강하게 요구하면 목표 명령을 무시하고 원래 속도를 고수할 수도 있다. 반대로 task만 강조하면 자연스러움이 나빠질 수 있다.

> **해결의 원칙**
>
> 보상 계수 하나를 크게 만드는 대신, 원하는 행동을 측정할 지표를 먼저 정한다. 실제 전진 속도·좌우 발 접촉·미끄러짐·몸 기울기·목표 오차를 나누어 보고, 이전 정상 조건도 함께 확인한다. 낙상 패널티만 키운다고 감속과 회복 동작이 자동으로 만들어지지는 않는다.

2부 · 실제 데이터

한 번의 rollout이 학습 자료가 되는 과정

예측한 모션이 아니라 실제로 적용한 action과 그 결과를 저장한다.

o_t → a_applied,t → physics → s_(t+1)
저장: 현재 관측, 실제 적용 action, pre/next state, done, goal, source

Teacher가 action을 내더라도 관절 범위를 넘으면 clamp가 적용될 수 있다. 이 경우 요청한 action과 실제 적용한 action은 다르다. 미래 sequence의 action 정답에는 실제 적용 값을 사용해야 다음 physical state와 맞는 인과관계가 된다.

상태 기록의 항목

필드	뜻
root_pos / root_rot	실제 몸통의 world 위치 / 회전 quaternion
root_vel / root_ang_vel	실제 선속도 / 각속도
q / dq	28 DoF 관절 각도 / 각속도
obs	policy가 그 순간 받은 실제 관측
applied_action	물리 엔진에 보낸 28차원 PD 목표
done / source / split	종료 여부, 원본 경험의 계보, 학습/검증 구분

정답이 바뀌면 데이터의 의미도 바뀐다

직접 action 모방의 일부 개발 단계에서는 student가 방문한 상태에 teacher가 제시한 action을 정답으로 붙였다. 하지만 H60 joint sequence 정답은 그 episode에서 **실제 적용된 action과 다음 상태**를 사용한다. Teacher label을 받았다는 것과 teacher가 실제 실행했다는 것은 구분해야 한다.

> **Reset 경계**
>
> 넘어져 reset한 다음 상태를 같은 연속 episode의 미래로 붙이면 로봇이 순간 이동하는 것을 정답으로 배운다. done 이전/이후를 구분하고 state continuity를 검사하며, 미래가 모자라는 window를 임의 padding하지 않는다.

2부 · 데이터 품질

Teacher 경험도 실행기로 재검사한다

모범 동작이 좋은 것과 공통 실행기가 그 동작을 실현할 수 있는 것은 별개다.

모든 성공 데이터를 그대로 쓰지 않은 이유

Teacher가 가진 제어 능력과 decoder가 가진 능력은 같지 않다. Teacher는 스스로 균형을 잡으며 만들었던 움직임이라도, 같은 root 특징을 본 decoder는 오차가 생길 수 있다. 좋은 데이터를 더 모으는 것만으로 생성–실행 간극이 사라지지는 않는다.

수집과 검사 흐름

- Teacher의 실제 움직임을 수집하고 goal·source·split을 보존한다.
- 공통 decoder가 사용할 미래 root 특징과 action·상태 정답을 구성한다.
- 고정된 decoder로 해당 경험을 실제 물리 실행해 usable 여부를 검사한다.
- 검사를 통과한 새 경험과 기존 정상 경험을 섞어 생성기·decoder를 보완한다.

최종 certified 데이터 기록에는 train29·validation8개의 triplet이 남아 있다. 하나의 triplet은 같은 source에서 갈라진 경험 묶음이며 독립 mocap29개라는 뜻이 아니다. 이 수와 window 수, source group 수는 다른 단위다. 모든 오래된 legacy 경험이 최신 decoder로 동일하게 재인증되었다고 가정하지 않는다.

같은 목표인데 다른 경로를 주는 이유

중간에 한쪽으로 가는 목표를 잠시 사용해 실제 우회 움직임을 만들고, 최종 목표에 맞는 입력으로 구성하는 개발 실험이 있었다. 이는 실행 시 모델에 waypoint를 준다는 뜻은 아니다. 다만 데이터 자체에 우회 경험을 설계·수집한 사람의 노력이 들어갔다는 점은 남는다. 목표를 바꾸어 만든 경험을 “아무 개입 없는 자발적 발견”으로 설명하면 안 된다.

> **Stop label**
>
> clip 끝이라는 이유만으로 stop이라고 붙이지 않는다. 실제 저속·저회전 유지가 있는지 본다. 목표를 지나 여러 번 빙빙 돌고 마지막에 성공한 경험과 첫 접근에서 감속·정지한 경험도 품질 관점에서는 다르다.

2부 · 학습 데이터의 수

많은 window가 많은 독립 경험은 아니다

같은 경로를 조금씩 잘라 만든 표본은 서로 닮아 있다.

최종 H60 dataset	값
Train windows	102,593
Validation windows	33,638
그중 새 검증 경험 window	train45,600 / validation19,500
한 window의 미래 길이	60 control step = 2초
초기 legacy episode당 최대 선택	500개 start index
새 episode당 최대 선택	300개 start index

예를 들어 10초 경로를 매 프레임마다 2초씩 잘라 학습하면 많은 window가 생긴다. 하지만 인접 window 대부분은 같은 움직임을 공유한다. 표본 수 10만이라는 말만으로 10만개의 독립 상황을 경험했다고 볼 수 없다.

Split과 정규화

동일 source의 crop·mirror·반복 rollout은 같은 split으로 묶어야 한다. H60 builder는 train/validation의 episode/source ID 겹침을 검사한다. Context와 첫 action 정규화는 부모 모델의 frozen train 통계를 상속하고, H60 나머지 target 통계는 train에서 다시 계산했다. Validation의 평균·표준편차로 train을 정규화하지 않는다.

최종 실험에서의 중요한 제한

학습 source split을 지킨 것과 평가가 완전히 새로운 시험이라는 것은 다르다. 최신 장애물 비교는 이미 개발에 사용한 validation source12개와 V1 장면을 재사용했다. 따라서 test-set 일반화보다 **고정 모델에 대한 개발 조건 비교**로 표현하는 것이 맞다.

> **재학습 배치의 균형**
>
> 최종 H60 학습은 새 경험이 batch의25%, 기존 경험이75%가 되도록 샘플링했다. 전체 dataset에서 새 window의 비율과, 배치에서 새 경험을 뽑는 확률은 같은 값이 아닐 수 있다.

2부 · 좌표와 단위

같은 동작을 같은 숫자로 표현하기

위치·회전·속도의 좌표가 어긋나면 모델보다 입력 변환이 먼저 문제다.

World와 현재 몸 기준 좌표

World 좌표는 시뮬레이터 바닥에 고정된 좌표다. 몸 기준 좌표는 현재 heading을 정면으로 본다. 같은 앞으로 걷기라도 world에서 북쪽·동쪽을 향하면 숫자가 달라지므로, 미래 state는 현재 root XY를 빼고 현재 yaw의 반대로 회전해 표현한다.

$$p_{local,xy} = R_z(-yaw_{now}) \times (p_{future} - p_{now})_{xy}$$
$$v_{local} = R_z(-yaw_{now}) \times v_{world}$$

Root 높이 z는 현재 높이를 빼는 대신 절대 높이를 유지한다. Quaternion은 x,y,z,w 순서를 사용하며, 6D 회전은 회전행렬 첫 두 column을 펼친 것이다. 회전 벡터를 변환할 때 단순히 숫자 순서만 바꾸면 안 된다.

왜 회전에 6개 숫자를 쓰나

각도를 직접 쓰면 −180도와180도가 사실상 같은 방향인데 숫자 차이는 크게 보인다. 6D 표현은 이 불연속을 줄이는 데 유리하다. 하지만 신경망이 낸 6개 값이 언제나 정확히 직교하는 단위 벡터는 아니다. 최종 root 특징에 이 표현 일부가 들어가므로 회전·속도 일관성 검사가 중요하다.

시간과 단위

항목	단위·해석
위치 / 선속도	m / m/s
관절 / 각속도	rad / rad/s
목표 방향 보고	degree로 표시하되 내부 계산은rad
Control / physics	30Hz / 120Hz, 제어당 물리 step4회
속도 차분	(다음 위치−이전 위치) / (1/30초)

World 목표는 실행 중 고정하고 현재 상태만 새로 관측한다. 매 재계획마다 목표를 현재 위치 기준으로 다시 멀리 옮기면 도착하지 않는 문제를 숨기게 된다.

2부 · 입력과 출력 계약

660개 입력에서 60×99개 미래 값으로

큰 숫자를 역할별로 쪼개면 구조가 단순해진다.

묶음	구성	차원
현재 관측 한 개	물리 관측140 + 목표4	144
최근 관측	t−3, t−2, t−1, t의 관측	144×4 = 576
과거 action	t−3, t−2, t−1의 실제 PD action	28×3 = 84
생성기 context	관측 이력 + 과거 action	660
미래 state 한 개	root3 + rot6 + v3 + w3 + q28 + dq28	71
미래 token 한 개	action28 + state71	99
미래 sequence	60 step의 token	60×99 = 5,940

물리140차원은 높이1 + 몸통회전6 + 선속도3 + 각속도3 + 관절회전84 + DoF속도28 + 주요5부위 상대위치15다. 회전은 heading 기준으로 표현한다. 목표4개는 상대XY와 방향 오차의 sin/cos다. 미래의 실제 관절이나 접촉 정보를 context에 넣어 정답을 누출하지 않는다.

과거 action을 넣는 이유

현재 자세가 같아도 직전에 어떤 관절 목표를 내었는지에 따라 관성·접촉·다음 반응이 달라질 수 있다. 짧은 상태 이력과 action 이력은 한 장의 자세만 보는 것보다 움직임 맥락을 제공한다. 초기 history가 부족할 때는 정의된 반복 관측·zero action 또는 저장한 prefix를 사용한다.

> **출력 전체를 실행하지 않는다**
>
> 현재 모델은 state-action sequence를 출력하지만 실제 장애물 실험의 명령은 공통 decoder가 만든다. 생성 action head는 첫8 step만 유효하게 학습하며, 뒤52 step action은 mask된다. 미래 state는60 step 전체를 예측한다. 5,940개가 모두 직접 제어 신호는 아니다.

2부 · MLP

MLP는 무엇을 학습했나

입력이 주어졌을 때 미래 움직임을 한 번의 forward로 예측한다.

회귀 모델의 직관

MLP는 여러 선형 변환과 비선형 함수를 쌓은 신경망이다. 현재 이력과 목표를 숫자 벡터로 넣고, 학습 데이터의 미래 sequence와 가까운 출력을 내도록 가중치를 조정한다. “MLP는 원래 부정확하다”는 명제는 맞지 않는다. 데이터·표현·학습 목표·실행 입력 분포가 맞으면 높은 성능을 낼 수 있다.

이 프로젝트의 실제 구조

660차원 context를1024→512 특징으로 변환하고,512 폭의 출력 네트워크가5,940개 값을 예측한다. 부모 direct-action policy를 frozen prior로 둔 anchored 구조이며, 첫 action에 작은 bounded correction이 붙는다. 총 parameter는5,199,184개다. 현재 실행은 action head 대신 state root 특성을 쓰므로 이 prior가 직접 장애물을 피하는 것은 아니다.

학습 정답과 손실

$$L = L_{state} + L_{root} + 0.05 \times L_{action} + 0.01 \times L_{consistency}$$

State와 root 손실은 정규화한 예측과 실제 미래 상태의 제곱 오차다. Root를 한 번 더 강조한다. Action 손실은 유효한 첫8 step을 사용하며 첫 action 비중은16/45로 조정했다. Consistency는 물리 단위에서 예측 위치 차분과 예측 선속도가 맞는지 본다.

평균화 문제가 생길 수 있는 조건

같은 입력에서 왼쪽·오른쪽 우회 같은 서로 다른 정답이 섞이면 제곱 오차 회귀는 중간 동작을 선호할 수 있다. 그러나 이 현상이 모든 MLP 실패의 원인이라고 단정하면 안 된다. 최신 대조에서는 출력 최적화가 물리적으로 이상한 경로를 만들던 별도 문제부터 발견하고 수정했다.

2부 · DIFFUSION

Diffusion은 왜 노이즈에서 시작하나

정답에 섞인 노이즈를 제거하는 연습을 통해 미래 움직임의 분포를 배운다.

학습

학습 때는 실제 미래 sequence에 크기가 다른 Gaussian noise를 섞는다. 네트워크는 현재 context와 noisy sequence, noise 단계 t를 받아 원래의 깨끗한 sequence를 예측한다. 이 구현은 noise 자체를 정답으로 삼는 ε-prediction 대신, 깨끗한 출력 x0를 예측한다.

```
x_t = sqrt(alpha_bar_t) × x0 + sqrt(1−alpha_bar_t) × noise
x0_hat = f_theta(context, x_t, t)
```

100단계 cosine schedule을 사용한다. 최종 학습에서는 배치512개 중 절반의 noise t를75~99에서 뽑고, 나머지는 전체0~99에서 뽑아 높은 noise 조건을 더 자주 학습했다. 이미지 생성과 같은 원리를 사용하지만 여기서 한 표본은 픽셀이 아니라 움직임의 숫자 sequence다.

생성

실행 때는 정답 미래를 알 수 없으므로 Gaussian noise 하나에서 시작한다. Denoising을 반복해 현재 상태·목표와 일치하는 미래를 만든다. 기본 후보 수 K는1이다. 여러 후보를 대량 생성해 외부 선택기로 성공한 것만 고르는 구조는 사용하지 않았다.

가능성과 한계

Diffusion은 같은 조건에 가능한 여러 미래를 표현할 수 있는 모델이다. 그렇다고 noise를 바꾸면 필요한 큰 우회가 반드시 나오는 것은 아니다. 실제 한 train context에서 noise8개를 바꾼 진단은1초뒤 횡방향 표준편차 약6.8mm를 보였다. 이 한 사례는 전체 분포의 증명이 아니지만, 다양성을 자동으로 가정하면 안 된다는 근거다.

최종 Diffusion은 context 특징과 noisy sequence·시간 embedding을 결합하며 총8,856,528 parameter다. MLP보다 parameter가 많아 같은 구조 크기의 순수 비교는 아니다.

2부 · DDIM과 모델 학습

16번의 복원으로 미래를 만든다

Diffusion 학습의100단계와 실행의16단계는 다른 숫자다.

DDIM 업데이트의 의미

```
noise_hat = (x_t − sqrt(alpha_t) × x0_hat) / sqrt(1−alpha_t)
x_next = sqrt(alpha_next) × x0_hat + sqrt(1−alpha_next) × noise_hat
```

실행 sampler는100단계 schedule에서16개의 시점을 선택해 결정론적 DDIM 형태로 이동한다. Noise 초기값과 context가 같으면 같은 출력을 재현할 수 있다. Sampling seed를 바꾸는 것은 새로운 네트워크를 학습하는 것과 다르다. [4]

최종 두 생성기의 학습 조건

항목	확인된 값
최종 update / seed	각8,000 / 학습seed0
Batch / optimizer	512 / Adam, 학습률1e−4→1e−5 cosine
Gradient clip / EMA	norm1 / EMA 비율0.005
Train windows	두 방법 모두102,593
모니터링 validation	균형 샘플2,048개
가중치 초기화	부모의 호환 가중치 상속, optimizer 새로 시작
학습 손실	state+root+0.05action+0.01consistency

최종 checkpoint는 validation state·root·consistency 기준으로 선택했다. Diffusion은 denoising validation 기준이 사용되어 sampled reconstruction과 같은 뜻이 아니다. MLP도 같은 코드 경로의 이름을 가진 선택 기준을 기록하지만 forward 자체는 결정론적이다.

> **학습 시간의 해석**
>
> 최종 H60 추가 학습 구간의 기록은 MLP약46.8초, Diffusion약71.4초다. 이는 이전 teacher 학습·데이터 수집·부모 모델 학습·실패 실험을 포함한 총 프로젝트 비용이 아니다. 가중치 load는 optimizer와 step까지 복원한 resume와 구분한다.

3부 · 구조 변화

왜 PPO tracker에서 decoder로 갔나

좋은 미래를 생성해도, 그 미래가 실제 action에 영향을 주어야 한다.

초기 reference tracker에서 배운 것

Reference tracker는 모션의 target 자세·속도를 관측해 실제 캐릭터가 따라가도록 PPO로 학습했다. 개별 모션이 잘 되어도 clip 전환에서 발 지지·속도·위상이 어긋나면 넘어졌다. 이를 단순 연결·위상 정렬·정지 보강·회복 학습 등으로 반복 보완했다.

그 뒤의 직접 action·joint sequence 시도

Tracker 의존을 줄이기 위해 teacher action을 직접 모방하는 MLP·Diffusion과 state-action sequence도 시험했다. 그러나 미래 root를 유도했는데 처음 실행할 action이 거의 바뀌지 않는 문제가 있었다. 상태와 action을 한 출력에 넣었다고 물리적 결합이 자동으로 학습되는 것은 아니었다.

현재 선택한 연결

미래 root 특징을 명시적인 조건으로 받는 **공통 learned action decoder**를 두었다. 생성기가 경로를 바꾸면 decoder가 그 조건을 보고 다음 관절 목표를 바꿀 수 있게 했다. 이 decoder는 최신 비교의 모든 방법에 동일하게 고정했다. 원래 PPO tracker와 동일 모델이나 PPO의 새 이름이 아니다.

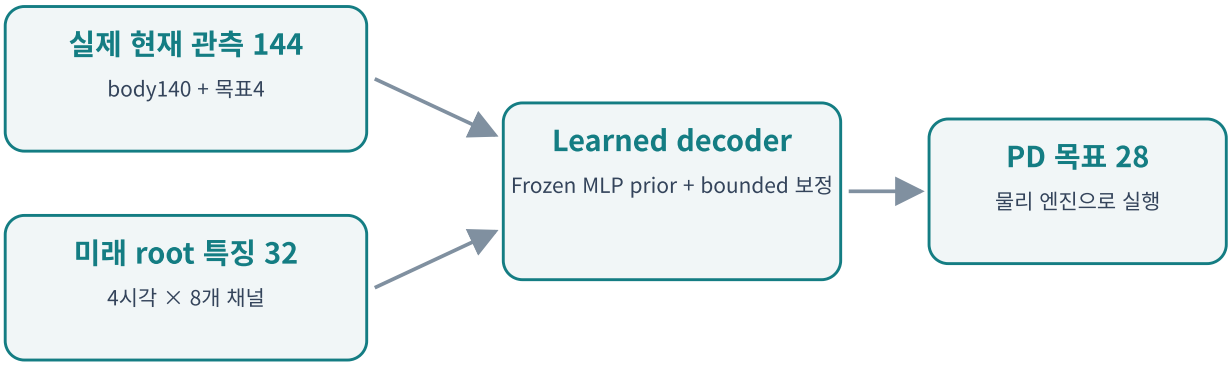
> **설명할 때 정확한 문장**
>
> “PPO·AMP로 teacher의 운동 경험을 만들고, sequence 생성기와 미래 root 조건 decoder를 지도학습했다. 실행은 그 decoder의 PD action으로 수행했다.”가 최신 결과에 맞다. “Diffusion이 만든 관절 모션을 PPO tracker가 그대로 따라갔다.”는 초기 구조와 최신 구조를 섞은 설명이다.

전체 state-action 출력의 일부 action head는 최신 실행에서 사용되지 않는 복잡성이 남아 있다. 성능 때문에 유지한 개발 구조이지, 반드시 필요한 최소 구조임을 입증한 것은 아니다. 후속 단순화 대상이 될 수 있다.

3부 · 고정 실행기

Decoder는 어떻게 관절 목표를 만드나

미래 움직임의 요약과 실제 현재 상태를 함께 본다.



생성된60 step 중 앞30 step(1초)만 decoder 특징에 사용한다. 시각 index는2,7,14,29로 약0.10·0.267·0.50·1.00초 후다. 각 시각에서 상대XY, 높이, 회전6D의 첫 column XY, 선속도XY, 각속도Z 등8개 채널을 꺼내32차원으로 만든다.

```
action = prior(actual_obs) + bounded_correction(actual_obs, future_root_features)
```

최종 checkpoint는 include_goal=true여서 보정 네트워크도 실제144차원 관측 전체와 root32를 함께 본다. 176→512→256→28의 보정망을 쓰고, action 공간에서 tanh로 최대0.5rad 규모의 보정을 제한한다. 총 parameter는916,792개다.

Decoder도 학습된 모델이다

기존 행동50%, 원래 branch25%, 새 teacher 우회25%의 혼합 배치로4,000 update를 보완했고 prior 가중치는 고정했다. 실제 teacher action을 정답으로 하는 지도학습이며 이 단계 자체가 새 PPO 학습은 아니다. 무작위로 root 값을 줘도 다 안정적으로 실현하는 보편적 dynamics inverse가 아니다.

3부 · CLOSED LOOP

미래를 길게 보되 짧게 실행한다

예측 오차를 실제 상태 관측으로 계속 수정한다.

한 번의 제어 cycle

- 현재 실제 obs와 최근 적용 action을 이력에 넣어660차원 context를 만든다.
- MLP 또는 Diffusion으로2초 미래를 생성하고 해당 방법의 비용 처리를 한다.
- 앞1초 root 특징32개와 현재 관측을 공통 decoder에 넣는다.
- 28차원 PD 목표를 한 control step 적용하고 물리 상태를 다시 읽는다.
- 같은 world 목표와 장애물을 유지한 채 다음 context로 다시 생성한다.

제어 step은1/30초이며 prediction horizon H=60, 실제 재계획 간격 E=1이다. 2초 동안 예측을 그대로 틀어 놓는 open-loop 방식과 다르다. 현재 상태가 예상과 다르면 다음 예측의 출발점도 달라진다.

PPO가 넘어지면 생성기도 이상해 보였던 이유

과거에는 tracker, 현재는 decoder의 실제 움직임이 다음 생성기의 관측으로 들어간다. 실행기가 뒤로 밀리거나 기울면, 생성기는 그 상태에서 미래를 다시 만든다. 그래서 원래 reference가 좋아도 실제 상태가 학습 분포 밖으로 가면 이후 reference도 흔들릴 수 있다. 두 모델의 가중치는 별개지만 피드백으로 동작은 연결되어 있다.

실시간과 simulation time

시뮬레이션 시간이30Hz로 진행된다는 것과, 실제 벽시계 시간에서 매번33.3ms 이내에 계산한다는 것은 다르다. 느린 계산을 기다리며 시뮬레이션을 진행할 수 있다. 최신 DP 유도 p50 약76.1ms는 planning+decoder만으로도33.3ms를 넘어, 본 결과를30Hz 실시간 제어라고 부르지 않는다.

> **오차의 출처를 나누는 법**
>
> 목표 방향 → 생성한 root 방향 → 실제 몸 방향을 시간별로 본다. 생성부터 틀렸으면 데이터·생성·유도 문제를, 예측은 타당한데 실제만 틀리면 실행 분포·접촉·지연을 의심한다. 시각적 인상 하나만으로 tracker 또는 생성기만 탓하지 않는다.

3부 · 장애물 정보

장애물을 모델에 어떻게 전달하나

신경망이 그림을 보고 판단하는 것이 아니라, 알려진 geometry로 비용을 계산한다.

입력 조건과 비용 조건을 구분한다

현재 생성기의 context에는 최근 몸 상태·action과 목표가 들어간다. 장애물의 위치·크기를 새로 학습한 관측 채널로 넣은 것은 아니다. 장애물은 시뮬레이터에서 얻은 원형 또는 회전된 상자 geometry로 표현하고, 생성 중 예측 위치가 가까워지는 정도를 비용 함수가 계산한다.

Signed distance: 얼마나 가까운가

SDF(point) > 0: 장애물 밖
SDF(point) = 0: 경계
SDF(point) < 0: 장애물 안

원형 장애물의 SDF는 중심까지 거리에서 반지름을 뺀 값이다. 상자는 점을 상자의 회전 좌표로 바꾼 뒤 직사각형 경계까지 거리를 계산한다. SDF를 쓰면 충돌 여부를0/1로만 보는 것보다, 어느 방향으로 떨어져야 비용이 줄어드는지 미분 신호를 얻을 수 있다.

안전 여유와 몸통 경로

계획 비용은 root의 평면 위치가 표면에서0.5m 안으로 들어오면 커지도록 했다. 이는 몸통 점 하나만 피하면 팔·다리도 안전하다고 가정하기보다 일정한 여유를 두려는 근사다. 다만 이 여유만으로 전신 충돌을 정확히 보장하지 못해, 실제 평가에서는 신체 부위와 접촉을 별도로 검사한다.

> **계획과 평가의 geometry 역할**
>
> 계획은 root 경로의 SDF로 비용을 만든다. 평가는 실제 body segment의 보수적 거리와 contact force proxy로 충돌을 판정한다. 계획의0.5m 여유와 평가의0.08m 충돌 문턱은 다른 항목이다. 계획이 장애물을 안 뚫어도 팔·발이 닿을 수 있다.

3부 · 비용 함수

무엇을 나쁜 미래라고 판단했나

충돌만 피하려다 멈추거나 비현실적인 움직임을 만들지 않도록 여러 항목을 함께 둔다.

$$J = 10 \times J_{collision} + J_{consistency} + 0.5 \times J_{course} + J_{progress} + 3 \times J_{preview} + J_{feasibility}$$

항목	계산 아이디어	필요한 이유
Collision	SDF0.5m미만 위험의 제곱 중 가장 큰5개 평균	근접·침범을 강하게 억제
Consistency	위치 차분과 예측 선속도 차이	좌표만 순간적으로 옮기는 편법 억제
Course	몸이 향한 방향과 선속도 방향 불일치	옆으로 끌거나 방향과 속도가 따로 노는 경우 억제
Progress	nominal 끝점 대비0.4m이상 진행 손실	제자리에서 멈추기만 하는 회피 억제
Preview	마지막 속도를 최대2초 연장한 예상 위험	2초 horizon바깥의 충돌을 조금 일찍 감지
Feasibility	속도·각속도·가속도 한계 위반	비현실적인 급변과 과속 억제

모든 항목은 학습된 reward model이 아니라 사람이 정의한 미분 가능한 비용이다. “Diffusion이 목적을 스스로 발견했다”는 표현보다 “비용으로 주어진 목적에 맞게 생성 과정을 조정했다”가 정확하다.

Preview의 한계

Terminal velocity preview는 현재 예측의 마지막 속도가 일정하다고 가정한 계산이다. 신체 접촉·관절 제약을 시뮬레이션한 실제 미래 rollout이 아니다. 먼 장애물을 조금 일찍 고려하는 데 도움을 줄 수 있지만, 회피 경로를 찾아 주는 전역 planner는 아니다.

V1의 task/settle 후보는 목표 진행·근거리 속도·위험 gate를 추가했으나 train에서 개선되지 않았다. 최신 V2 주 비교는 기존 route-free 비용에 동일 feasibility 항목을 더한 방식이다. 실패한 후보를 선택된 최종 방식처럼 설명하지 않는다.

3부 · 생성 중 유도

비용이 Diffusion을 어떻게 바꾸나

나쁜 결과를 만든 원인을 noisy sequence 쪽으로 거슬러 올라간다.

Gradient의 직관

비용 J가 커지는 방향을 gradient라고 한다. Gradient의 반대로 조금 움직이면 비용을 낮추는 방향으로 입력이 바뀐다. Diffusion에서는 깨끗한 예측 x0_hat에서 비용을 계산하고, 그 비용을 denoiser를 통과해 현재 noisy sequence x_t까지 미분한다. 네트워크 가중치는 바꾸지 않는다.

```
g = d J(physical(f_theta(context, x_t, t))) / d x_t
x_t ← x_t − scale × sqrt(1−alpha_t) × g / max(||g||, tiny)
```

선택된 V2는 unit-gradient 형태와 guidance scale15를 사용한다. 각 denoising 시각에서 유도 업데이트를1회 수행하고, 수정한 noisy sequence를 다시 denoise한다. 16단계마다 가중치 학습 없이 이 작업을 반복한다.

한 장면의 생각 흐름

현재 예상 미래가 원통 쪽으로 가면 SDF 비용이 커진다. 그 비용이 감소하도록 noisy sequence를 바꾸고 복원하면 미래 root 경로와 관련 state가 달라질 수 있다. 고정 decoder는 바뀐 미래 root 특징을 조건으로 관절 명령을 바꾼다. 실제 몸이 옆을 통과하면 그 실제 위치가 다음 생성의 입력이 된다.

왜 후처리와 다른가

이 방식은 최종 출력 root를 따로 잘라 옆으로 이동시키지 않는다. 비용을 모델의 복원 과정에 전달해 학습된 생성기의 출력을 바꾼다. 다만 비용이 강하면 학습 분포에서 벗어나거나 일관성이 나빠질 수 있으며, 항상 실행 가능한 결과를 얻는다는 보장은 없다.

> **여전히 필요한 사람의 설계**
>
> 비용 항목·계수·geometry·실행기·도착 보조는 사람이 정했다. 비용 유도가 연구적으로 의미 있다는 것과 human effort가 사라졌다는 것은 다르다. 이 프로젝트는 후자를 입증하지 않는다.

3부 · 세 방식의 구분

유도·출력 최적화·root 보정

어디에서 숫자를 바꾸는지에 따라 결과의 의미가 달라진다.

방식	숫자를 바꾸는 위치	경로 지식	최신 역할
Diffusion 비용 유도	denoising 중 noisy sequence	SDF·동작 비용, 지정 우회선 없음	주방법
MLP 출력 최적화	모델이 낸 뒤 root 경로 변수	같은 SDF·동작 비용	주 대조군
Analytic root 보정	생성 후 root를 회전·재구성	장애물 geometry로 우회방향·곡선 계산	별도 외부 기준선

Root 보정

Root는 몸통의 위치·방향을 대표한다. Analytic 보정은 장애물 위치에 따라 돌아갈 방향과 경로 변형을 계산하고, 이미 생성한 root 경로를 그쪽으로 바꾼다. 계산이 빠르고 이번24장면에서24/24였지만, 그 성과 전체를 Diffusion의 판단 능력으로 돌리면 안 된다.

곡선 경로 힌트를 준 유도

이전 D2 비교는 denoising 안에서 비용을 적용했지만, 비용에 geometry로 만든 곡선 target도 들어 있었다. 따라서 “출력 후 root 보정 없음”은 맞아도 “외부 우회 경로 지식 없음”과 같지 않다. 이를 분리하려고 V1·V2에서는 해당 target과 side 선택을 없앴다.

현재 주방법에 남은 것

Root 경로를 사후 변형하는 보정과 analytic 우회 곡선은 없다. 고정 decoder, 목표 방향의 이동/정렬 단계 전환, joint action clamp는 공통으로 남았다. “순수 Diffusion 하나로 관절까지 직접 제어했다”거나 “아무 실행 보조가 없다”는 표현은 피한다.

> **세 방법 모두 보고하는 이유**
>
> Root 보정이 강하다는 사실을 숨기면 Diffusion 역할을 과장하게 된다. 주방법은 경로 힌트 없이도 얼마나 해결하는지 보여주고, 별도 보정은 더 단순한 방법이 얼마나 잘하는지 보여주는 비교다.

3부 · MLP 대조 보완

0/24 실패를 그대로 결론으로 쓰지 않았다

목표 비용을 줄이는 것과 물리적으로 말이 되는 경로를 만드는 것은 다르다.

V1에서 발견한 문제

이전 MLP 대조는 매 future frame의 XY와 yaw를 자유롭게 수정하는 Adam 최적화였다. 곡선 target을 제거하자 좁은 시간에 위치·방향이 크게 달라지는 경로가 나왔다. 첫 실제1초의 예측 속도p95를 episode별로 평균하면 약8.64m/s, 회전속도45.63rad/s였다. 이 값은 실제 로봇 속도가 아닌 생성 계획의 값이다.

잘못된 결론을 피한다

이 대조의0/24는 “MLP는 모션 생성에 쓸 수 없다”의 근거가 아니다. 제약 없는 optimizer가 모델의 출력 공간을 너무 자유롭게 움직인 문제일 수 있었다. 그래서 최종 비교 전에 과도한 위치 변화가 생기지 않는 parameterization을 만들었다.

V2: 가속도를 적분해 root를 만든다

$$a(t),\ \text{yaw_accel}(t) = \text{9개 knot를 시간 보간한 제어값}$$
$$v(t) = \text{bound}[v_{\text{initial}} + dt \times \text{cumulative_sum}(a)]$$
$$p(t) = dt \times \text{cumulative_sum}(v)$$
$$w(t) = \text{bound}[w_{\text{initial}} + dt \times \text{cumulative_sum}(\text{yaw_accel})]$$
$$\text{yaw}(t) = dt \times \text{cumulative_sum}(w)$$

시작 속도는 실제 현재 몸 속도를 local frame으로 바꾼 값이다. Knot는 경로 위의 waypoint가 아니라 시간에 따른 제어 신호의 대표값이다. 좌우 우회 쪽이나 중간 목표 위치를 지정하지 않는다. root 높이·나머지 state는 nominal을 보존하고 방향·평면 속도 관련 부분을 일관되게 다시 구성한다.

> **이 보완의 수준**
>
> Root의 운동학적 제한을 만들었을 뿐, 접촉·토크·관절 균형까지 만족하는 전신 최적제어를 푼 것은 아니다. 그래서 물리 실행으로 결과를 다시 평가해야 한다.

3부 · 비교의 공정성

같게 맞춘 것과 다른 것

동일 정보 비교와 순수한 모델 구조 비교는 서로 다른 실험이다.

맞춘 조건	설명
정보	동일 실제 상태·이력·목표와 장애물 geometry
데이터	같은 H60 dataset, train-only 정규화
실행	같은 frozen decoder·물리 backend·H60/E1
비용	같은 SDF·진행·일관성·방향·preview·feasibility
평가	같은 장면·초기 상태·최종 목표·성공/충돌 기준
설정 선택	각 방법 train2후보로 선택 후 고정

남은 차이	해석 제한
MLP vs Diffusion 크기	약520만 vs886만 parameter
최적화 공간	MLP: bounded root 제어 / DP: noisy sequence
제약 방식	MLP hard bounds / DP 같은 한계값의 soft penalty
추론 반복	MLP Adam32회 / DP DDIM16회+유도
모델 계보·학습 목적	부모 prior가 다르고 Diffusion은 denoising 학습
sampling	MLP 결정론적 / DP seed2개 모두 평가

따라서 “같은 장애물 문제를 같은 정보와 실행기 아래에서 두 방식으로 풀어 본 시스템 비교”는 가능하다. 하지만 차이를 Diffusion 구조 하나에만 귀속하는 architecture 인과 실험이라고 설명할 수 없다.

Train에서 선택한 값

MLP learning rate .05/.15의 성공은3/12·2/12로 .05를 골랐다. Diffusion scale15/30의 성공은7/12·6/12로15를 골랐다. 성공 수를 먼저, 이후 낙상·충돌을 보고 동률이면 먼저 등록한 후보를 고르는 순서를 사용했다. 최종 장면을 본 뒤 후보를 추가하거나 성공 문턱을 바꾸지 않았다.

4부 · 실험 설계

무엇을 몇 번 평가했나

장면·source·sampling seed·학습 seed를 분리해서 센다.

항목	V2 조건
장면	24개: 원형8·상자8·벽8
Source	개발 경험12개, V1배치 재사용
시작	원래 snapshot에서0.5초 뒤 실제 보행 상태
목표 위치	거리5~6.5m, 방위−25/0/+25도
장애물 위치	정면2.5~3.5m, 중심선에서좌우약0.2~0.45m
장애물 형태	원형폭0.4/0.8/1.2m, 상자최대폭0.8/1.2m, 벽1.4/1.8m
도착 방향	초기 몸 heading과 같은 방향
Learning seed / sampling seed	학습1개(seed0) / Diffusion4504·4505
Physics seed	4504 고정

MLP와 root 보정 기준선은 장면당1회, Diffusion 유도는 두 sampling seed를 모두 실행했다. 따라서 작은 범위의 DP23/24는 작은 장면24종이 아니라 **12장면×2sampling seed**다. 작은 범위·스트레스 범위는 각각12장면과8개 source group이다.

실행 비용의 정확한 집계

Train48회 + 후속120회 = 새 scored168회, 별도 단일 환경 profile5회로 새 물리 실행은173회다. 유도 없는 DP48회는 동일 조건의V1 저장 결과를 재사용했다. 추가 가중치 학습은0회다. Subprocess walltime 합은약1,719.1초이며 최대2process 병렬 실행의 합이므로 전체 경과 시간이나 GPU 단독 사용 시간과 다르다.

> **일반화 범위**
>
> 이전 개발에서 본 source와 장면을 재평가했으므로 완전히 보지 못한 test 환경에 대한 성능으로 소개하지 않는다. 정중앙을 대칭으로 막는 장애물·여러 장애물·180도 목표·정지 출발의 전체 navigation은 이 평가에서 검증하지 않았다.

4부 · 성공 판정

어느 정도면 도착해 멈췄다고 하나

목표를 잠깐 통과한 것과 안정적으로 도착한 것을 다르게 센다.

실제 상태 기준	V2 고정 문턱
목표 위치 오차	1.25m 이하
최종 방향 오차	45도 이하
평면 이동 속도	0.15m/s 이하
몸통 Z 각속도 절댓값	0.2rad/s 이하
유지 시간	위 모든 조건을 연속5초
유지 구간 drift	5초 구간 시작~끝 순변위0.1m 이하
추가 확인	성공 interval 후2초 동안 낙상 없음
전체 실행 상한	45초, 충돌이 한 번이라도 있으면 주 성공 제외

허용 범위를 비교적 넓게 둔 것은 이 과제가 정밀 발 배치·정밀 도킹보다 대략적인 도착·정지에 초점을 두기 때문이다. 그렇다고 속도·유지·충돌 조건까지 없어지는 것은 아니다. 결과를 본 뒤 문턱을 완화한 것이 아니라, V2에서는 이전에 고정한 V1 기준을 그대로 썼다.

세 가지 서로 다른 조건

훈련 보상은 좋은 행동을 배우게 하는 연속 점수다. **실행 supervisor**는 언제 최종 방향 요청으로 전환하는지를 정한다. **평가 성공**은 실제 결과가 문턱을 만족했는지 판단한다. 평가에서 성공이라고 선언했다고 정지 action을 대신 넣는 것은 아니다.

판정의 세부 한계

Drift는 구간 시작과 끝의 차이다. 구간 중 최대 흔들림 반경을 뜻하지 않는다. 실제 낮은 속도 문턱도 함께 있지만, 왕복·원형 움직임을 이 항목 하나로 완전히 막지는 못한다. 또한 성공 후2초 조건은 낙상 없음이며, 그2초 내내 모든 정지 문턱을 재충족하도록 정의한 것은 아니다. 이 의미를 그대로 보고한다.

4부 · 충돌과 낙상

피한 것처럼 보여도 접촉했을 수 있다

실제 접촉 정보와 보수적인 신체 거리 검사를 함께 사용한다.

물리적 장애물

원형은 collision cylinder, 상자는 회전 가능한 rigid box로 실제 시뮬레이터에 추가했다. 영상에 표시한 장식물을 지나가는 평가가 아니다. 움직임은 장애물과 실제로 상호작용하며, 몸이 막히고 기울고 넘어질 수 있다.

Contact force proxy

현재 backend는 장애물 전용 contact sensor를 따로 만들지 않아, 캐릭터 전체 접촉력에서 지면 접촉력을 뺀 뒤 합력의 크기를 사용한다. 이 값이1N보다 크면 장애물 접촉으로 판정한다. 합력은 서로 다른 접촉이 상쇄될 수 있고 정확한 mesh 접촉 쌍을 직접 식별하는 센서가 아니므로 proxy라고 부른다.

Body segment geometry 검사

신체 부위와 부모 부위를 잇는 segment를33개 점으로 샘플링해 obstacle SDF를 계산한다. 샘플 사이를 놓칠 가능성에 대비해 spacing에 따른 Lipschitz 보정을 빼 보수적인 clearance 하한을 만든다. 높이1.5m의 장애물과 겹칠 수 있는 신체 부분을 고려한다.

충돌 = [장애물 contact force proxy > 1N]
OR [body surface clearance lower bound < 0.08m]

보수적 판정은 정확한 mesh 접촉이 없더라도 너무 가까운 움직임을 실패로 둘 수 있다. 그래서 실제 force 접촉과 conservative collision 수를 구분해 기록한다. 최신 DP는 실제 접촉13/48, 보수적 충돌14/48이었다.

낙상과 timeout

낙상은 환경의 FAIL 종료로 판단했다. Timeout은45초 안에 성공 기준을 끝내지 못한 경우다. 벽에 오래 막힌 DP scene2는45초 timeout이며, 끝에 넘어졌다고 임의로 해석하지 않는다. 충돌과 낙상은 동시에 일어날 수 있으므로 실패 수를 단순히 더하면 중복된다.

4부 · 전체 결과

동일 비용 아래에서 무엇이 달랐나

비교를 수행한 것과 모든 방법을 개선한 것은 다르다.

방법	성공	낙상	실제 접촉
MLP 원본	8/24	8/24	13/24
MLP 제약 출력 최적화	9/24	10/24	13/24
DP 원본, 동일 조건 재사용	0/48	29/48	48/48
DP 비용 유도, 경로 힌트·후처리 없음	33/48	7/48	13/48
별도 analytic root 보정	24/24	0/24	0/24

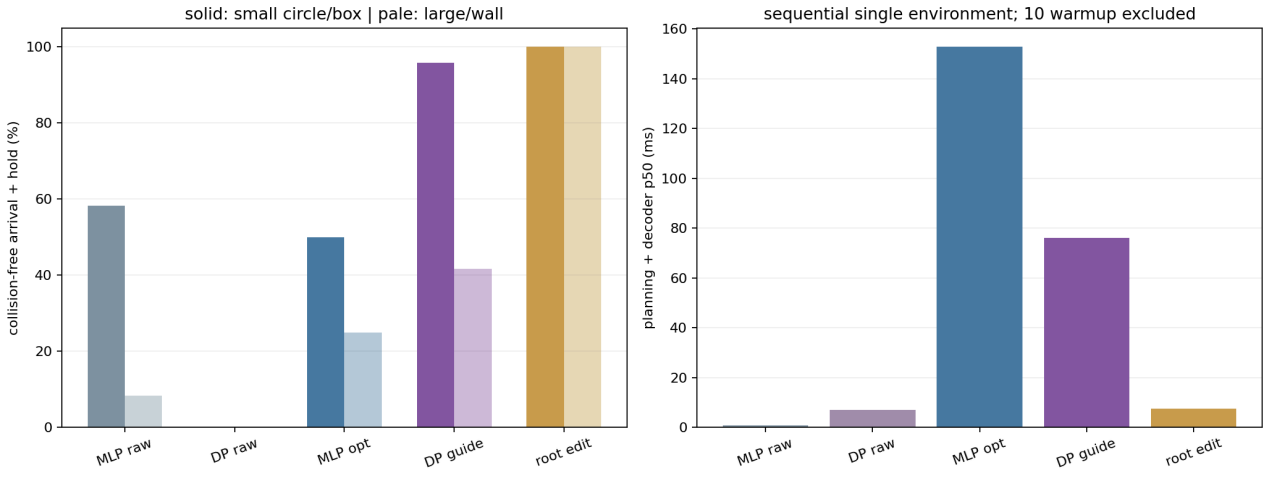


그림 1. 작은 장애물(진한 막대)과 큰 장애물·벽(연한 막대)의 성공률, 단일 환경 planning+decoder p50.

같은 source 단위로 MLP와DP 점수를 비교한 차이는31.25%p다. Source12개를 묶음으로 재표집한95% bootstrap 구간은약[4.2,56.2]%p였다. 같은 source에서 나온 장면과 sampling seed를 전부 독립 표본처럼 취급하지 않으려는 분석이다.

하지만 이 구간은 고정된 checkpoint와 개발 장면의 불확실성만 일부 다룬다. 여러 학습 seed를 다시 학습한 결과가 아니므로 학습 재현성이나 architecture 일반 우월성의 구간으로 해석하지 않는다.

4부 · 성공 범위

작은 장애물에서는 강하고 벽에서는 약하다

범위를 나눠야 전체 평균 뒤에 숨은 차이를 볼 수 있다.

시험한 최대 평면 폭	제약 MLP	DP 유도
원형0.4m	1/3	6/6
원형0.8m	3/3	6/6
원형1.2m	0/2	2/4
상자0.8m	2/6	11/12
상자1.2m	1/2	2/4
벽1.4m	0/4	2/8
벽1.8m	2/4	4/8

무엇을 말할 수 있나

원형·상자의 최대 폭0.8m 이하를 사전에 작은 조건으로 정했다. 이 범위에서DP는23/24, MLP는6/12였다. 더 큰 물체·벽에서는DP10/24·MLP3/12로 낮아졌다. 따라서 주 데모를 작은 장애물에 두고 넓은 벽은 스트레스 평가로 보여주는 것이 결과에 맞다.

무엇을 말하면 안 되나

폭1.8m 벽이1.4m보다 숫자가 높다고 해서 큰 벽이 더 쉽다는 결론을 내릴 수 없다. 각 bin의 시작 상태·장애물 위치·각도가 다르고 표본 수도 작다. 같은 배치에서 크기만 하나씩 바꾼 인과 실험이 아니므로 크기별 단조 관계를 주장하지 않는다.

> **23/24의 제한**
>
> 12개의 작은 장면에 sampling seed2개를 적용한 값이다. 완전히 새로운 시작 자세·중앙 장애물·반대 방향 목표에 대한95.8% 보장도 아니고, 충돌 없는 경로가 항상 생성된다는 안전 인증도 아니다. 성공한 장면이 무엇인지와 실패가 어디 남는지를 함께 보여주는 경험적 범위다.

4부 · 계산 시간

잘하는 것과 제시간에 계산하는 것

Physics time, batch time, 실제 단일 환경 latency를 구분한다.

방법	p50	p95
MLP 원본	0.9ms	1.2ms
Diffusion 원본	7.1ms	8.2ms
MLP 제약 최적화	152.7ms	180.9ms
Diffusion 비용 유도	76.1ms	81.5ms
Analytic root 보정	7.6ms	8.5ms

RTX4060Ti8GB에서 한 환경씩 순차 실행하고 CUDA synchronize 후 측정했다. 최초10회는 warmup으로 제외했고 각 방법110개 query를 사용했다. Scene0의 처음4초 동안 planning+decoder 구간을 재었으며 physics step·관측 수집·logging은 포함하지 않았다.

왜 MLP 최적화가 느린가

MLP forward 자체는 빠르다. 하지만 출력 경로를 Adam으로32번 고치면서 cost와 gradient, candidate 비교를 계산하면 추가 시간이 든다. 이 문제는 “MLP 신경망이 원래 Diffusion보다 느리다”의 증거가 아니다. 어떤 추론 알고리즘을 붙였는지까지 포함한 시간이다.

비용 유도의 시간

DP 유도는16회 denoising에 gradient 계산까지 들어간다. 이번 설정은 MLP 출력 최적화보다 빠르지만30Hz의33.3ms 예산은 넘는다. 따라서 화면에서 부드럽게 보이는 영상을 실제 벽시계 시간30Hz 실행의 증거로 쓰면 안 된다.

> **측정의 한계**
>
> 4초·장면1개·고정 장치의 측정이다. 전체 장면의 최악 지연·멀티 환경 속도·GPU 동시 사용 조건을 대표하지 않는다. 같은 walltime 예산을 준 최적화 비교도 아니다. 영상 overlay의 per-step latency는 병렬 batch 측정값으로 별도 표기했고, 공식 비교는 이 단일 환경 값이다.

4부 · 실패 진단

실패를 생성과 실행으로 어떻게 나눴나

접촉 전에 모델이 내다본 미래부터 확인했다.

재현 가능한 진단 절차

- 첫 실제 contact 시각을 찾는다. 없으면 첫 보수적 geometry 충돌 시각을 쓴다.
- 그0.5초 전 생성한 예측의 첫1초 root 경로를 실제 world 좌표로 옮긴다.
- 그 경로의 최소 SDF가 음수면 이미 장애물 내부를 계획한 것으로 분류한다.
- 0~0.5m면 안전 여유 부족,0.5m이상이면 신체/실행 오차 의심으로 둔다.
- 접촉 없이 넘어진 경우 생성·실행 원인을 확정하지 않는다.

실패 유형	MLP 최적화	DP 유도
예측 root가 장애물 내부	1	8
예측 경로 여유0.5m미만	10	6
root 여유는 있으나 신체/실행 접촉 의심	3	0
접촉 없는 낙상, 원인 미확정	1	1
총 실패	15/24	15/48

DP 실패15개 중14개는 생성 경로 자체에서 위험 신호가 있었다. 따라서 모든 실패를 “안전한 reference를 decoder가 못 따라가서”라고 설명할 수 없다. MLP에는 root 경로가 비교적 떨어졌는데도 신체 접촉한 사례3개가 있어, 전신 shape와 실행 오차를 추가로 봐야 한다.

> **인과관계의 한계**
>
> 0.5초 뒤 실제 실행은 재계획으로 바뀔 수 있고, root는 전신 모양을 대표하지 못한다. 이 분류는 관찰을 일관되게 정리한 진단이다. 해당 예측을 바꾸면 낙상이 반드시 사라진다는 counterfactual 검증이나 원인의 배타적 확정은 아니다.

4부 · 장면 분석

같은 원통, 유도 전후의 차이

출발 상태가 같아도 생성 결과와 이후 실제 상태는 달라진다.

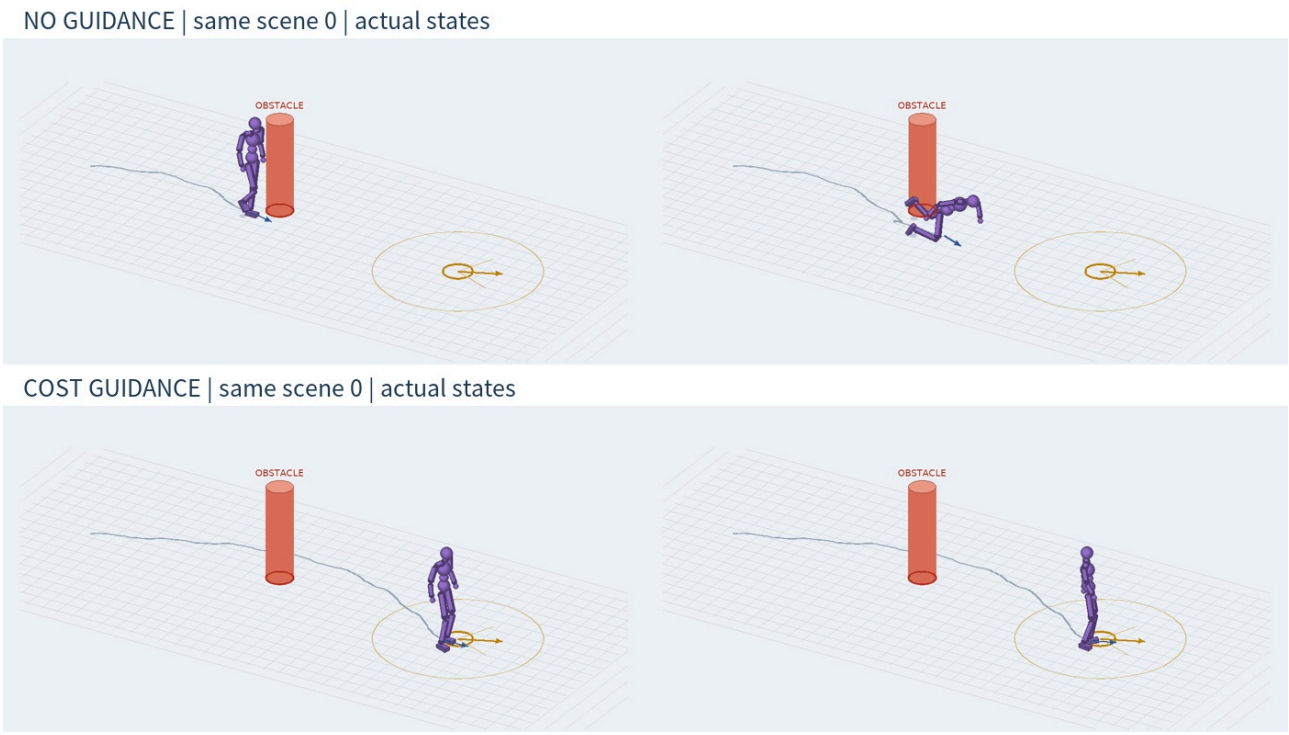


그림 2. 같은 scene0의 실제 물리 상태. 위: 유도 없는 DP가 원통에 막힌 뒤 약6.6초에 낙상. 아래: 비용 유도 DP가 우회해 목표 근처에서 정지.

유도 없는DP는 원통 쪽으로 진행했고 접촉 뒤 무너졌다. 비용 유도DP는 원통 옆으로 움직여 지나간 뒤 목표 방향과 낮은 속도를 유지했다. MLP 제약 최적화도 이 장면에서는 성공했다. 따라서 이 영상 하나로 “MLP는 실패하고DP만 성공한다”고 설명하면 안 된다.

예측 선을 읽는 방법

분석 영상의 회색 선은 그 순간 실제 context에서 유도 없이 만든 nominal root, 보라색은 같은 context와noise에서 비용을 적용한 예측이다. 파랑은 실제 실행 궤적이다. 이는 그 순간의 변화를 보여준다. 반면 raw run과 guided run 전체는 시간이 지나면서 actual state가 달라지므로, 매 순간 같은 context를 유지한 두 실험으로 보지는 않는다.

Scene0 비용 유도는 약15.3초에 episode를 끝냈다. 성공 기준의5초 유지와2초 post 확인도 포함한 시간이며, 순수 이동 시간만 뜻하지 않는다.

4부 · 장면 분석

같은 상자, MLP와 Diffusion

장면4는 방법별 차이가 눈에 보이는 사례지만 전체 평균을 대신하지 않는다.

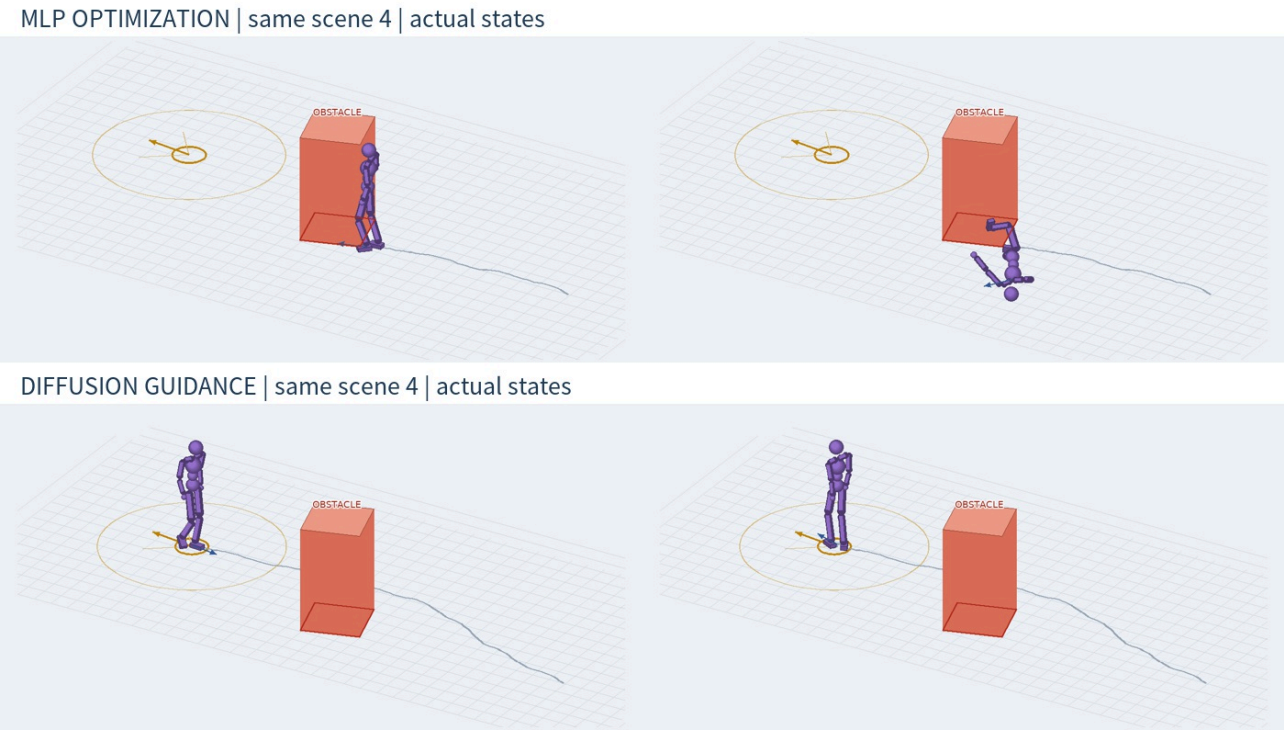


그림 3. 위: 제약 MLP가 상자 옆에서 막힌 뒤 약8.7초에 낙상. 아래: 같은 시작 상태의DP 유도가 상자를 지나 목표에 도착한 실제 상태.

MLP는2~3초 무렵 상자에 가까워졌고, 예측 경로의 root clearance가 줄어들었다. 접촉 후 속도가 낮아졌지만 목표에서 약3.5m 떨어진 상태에 머물렀고 결국 넘어졌다. Root 속도·가속도를 제한했어도 우회 여유와 실제 균형이 충분하지 않았다.

Diffusion 유도는 상자를 피해 진행했다. 약6.7초에는 목표 근처로 들어왔고, 뒤에는 작은 움직임과 방향 오차를 유지하며 정지 조건을 완료했다. 같은 장면에서는DP가 더 잘했지만, 두 모델의 학습 목적·parameter·최적화 parameterization이 다르다는 비교 한계는 그대로다.

대표 사례를 고른 규칙

Seed4504에서 ID순 첫 작은 장애물DP 성공을 기본 사례로, 첫DP 실패를 실패 사례로 골랐다. 추가로 작은 장애물 중 첫 MLP 실패·DP 성공 장면의 쌍을 골랐다. 성공만 선별해 전체 성능처럼 보여주지 않고, 모든 episode는CSV에 남겼다.

4부 · 시간별 진단

막히기 전부터 계획 여유가 줄었다

영상의 느낌을 시간별 수치와 연결한다.

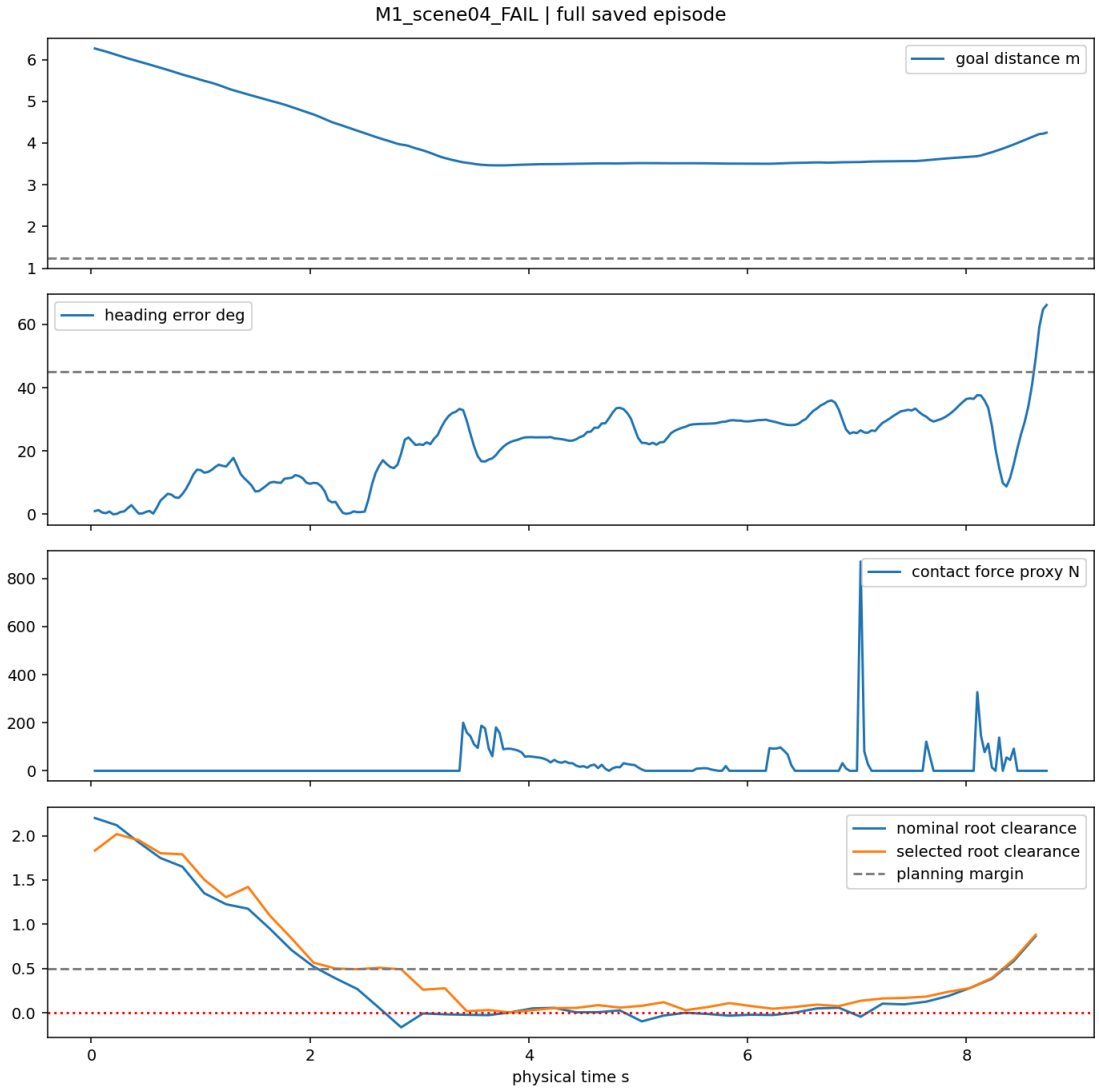


그림 4. MLP scene4 전체 기록. 위치 오차, 방향 오차, 접촉력 proxy, root 예측 최소 여유를 시간축으로 정렬했다.

마지막 패널에서 selected root 여유가0.5m아래로 내려간 뒤 접촉이 시작되는 흐름을 볼 수 있다. 접촉 후 목표 거리가 줄지 않고 거의 일정하며, 끝에는 방향 오차와 위치 오차가 커진다. “정지 모션이라서 못 섰다”만으로 설명하기보다 **장애물 접근→여유 부족→접촉·정체→낙상**의 흐름으로 보는 것이 이 장면에 맞다.

지표의 선후관계는 가능한 실패 메커니즘을 좁혀주지만 원인의 확정은 아니다. 예측 모델의 위험을 낮춘 새 시도와 실행 결과까지 비교해야 개입 효과를 말할 수 있다.

4부 · 남은 실패

벽은 왜 계속 밀고 있었나

큰 우회를 요구하는 상황에서 국소 비용만으로 경로가 충분히 바뀌지 않았다.



그림 5. DP scene2 실제 실행. 벽에 접근한 뒤 예측 root가 벽 근처를 계속 향하며 정체했다. 영상은20초까지만 보여주지만 전체 episode는45초 timeout이다.

관찰

약6.7초부터 몸이 벽에 닿은 채 속도가 거의0으로 줄었다. 그 뒤에도 생성된 root는 벽을 충분히 돌아갈 여유를 확보하지 못했다. 안전하게 서서 목표에 도착한 것이 아니라 장애물에 막혀 목표 약3.17m앞에서 움직이지 못한 상태다.

가능한 설명

짧은 horizon과 국소 gradient, 학습 데이터의 우회 다양성, 낮은 nominal 다양성이 함께 작용했을 수 있다. 넓은 벽은 현재 경로에서 옆으로 크게 벗어나고 일시적으로 목표 진행을 늦추어야 한다. 진행 보상과 nominal 분포가 직진을 선호하면 그런 경로로 이동하기 어려울 수 있다. 이 설명은 아직 원인별 ablation으로 분리한 결론은 아니다.

다음 개선의 방향

유도 강도를 계속 키우기보다 train에서 같은 최종goal의 좌우 우회 경험과 생성 다양성을 확인하고, 실행 검증된 자료가 부족하면 보강하는 것이 우선이다. 더 긴 horizon·다른 비용·전역 계획은 가능한 후속이지만 이번 성과로 완료 처리하지 않는다.

4부 · 개발 과정

처음부터 최종 구조까지의 흐름

반복 실험은 어떤 연결이 부족한지 찾는 과정이었다.

단계	관찰한 문제	이어진 판단
G1 속도 실험	속도 명령을 바꿔도 원래 보행에 머묾	Task와 reference 영향 분리, 이후 generic humanoid로 범위 정리
다섯 모션 PPO 추종	개별 모션은 되지만 전환에서 발·속도 불연속	전환 정렬·정지 유지·실제 추종 점검
명령/목표 PPO+AMP	정지·우회전·큰 방향 전환의 편향	Goal teacher·단계적 접근/정렬과 품질 보완
검색·reference MLP/DP	jitter·목표 overshoot·정지 중 낙상	actual feedback·입출력/시간 계약·데이터/실행 간극 진단
직접 action / joint sequence	미래 유도가 첫 action에 약하게 반영	명시적 미래 root 조건 decoder 도입
장애물 유도	analytic 보정은 강하지만DP기여 모호	경로 hint·출력 root 보정 제거 후 별도 검증
Guidance-only V1/V2	MLP optimizer 폭주·큰 장애물 충돌	제약 MLP 대조·범위 구분·예측/실행 실패 진단

이 표는 주요 설계 판단을 설명하는 흐름이다. 모든 과거 모델을 최종 실험에 합산하지 않는다. 각 단계가 같은 모델의 완전한 optimizer resume로 이어진 것도 아니다. 부모 가중치 상속, 새 optimizer, 별도 teacher·tracker·decoder 학습이 섞여 있으므로 모델 계보는 파일별로 구분한다.

사용자는 실제 영상을 확인하며 발이 튀는 전환, 정지 시 뒤·앞 낙상, 과도한 허용 조건, 작은 task만 평가하는 문제를 지적하고 방향을 결정했다. 코드 작성·분석·반복 실행·문서에는AI 도움을 사용했다. 외부 알고리즘과 직접 추가한 부분의 구분도 프로젝트 설명에 포함한다.

4부 · 결과의 의미

무엇을 기여라고 설명할 수 있나

새 알고리즘이라는 주장보다 검증된 시스템과 분석을 선명하게 설명한다.

확인한 기여

- 기본 모션·강화학습 경험을 실제 데이터로 만들고 미래 생성과 물리 실행까지 연결했다.
- 생성된 미래와 action의 결합이 약한 문제를 발견해 공통 root-conditioned decoder로 명시적인 연결을 만들었다.
- 장애물 비용 유도, 출력 최적화, analytic root 보정을 구분하고 같은 조건에서 비교했다.
- 잘못된MLP대조를 그대로 사용하지 않고 속도 폭주를 진단·보완했다.
- 작은 장애물의 높은 성공과 큰 장애물의 실패를 나눠 보고, 생성 계획과 실제 실행을 함께 분석했다.

주장하지 않는 것

비용 유도나PPO·AMP를 새로 제안했다고 하지 않는다. Diffusion이 반드시 필요한 문제라고 증명하지 않았고, 소수 모션만으로 모든 locomotion 기술을 무데이터로 얻었다고도 하지 않는다. 실제 로봇 적용·sim-to-real·실시간·독립source일반화·human effort절감을 검증하지 않았다.

왜 여전히 포트폴리오로 의미가 있나

핵심은 영상 하나가 아니라 실패를 측정하고 잘못된 해석을 고치며 비교가 가능한 구조를 만든 경험이다. 로봇 학습에서는 데이터의 모양, 좌표, 시간, 실행 오차와 평가 정의가 성능을 크게 바꾼다. 이 프로젝트는 그 연결을 직접 다뤘다는 근거를 가진다.

> **최종 소개 문장**
>
> 강화학습으로 수집한 휴머노이드 운동 경험을MLP와Diffusion으로 모델링하고, 추가 학습 없이 비용으로 장애물 회피를 유도했다. 경로 힌트·후처리 없는DP는 작은 장애물23/24, 전체33/48 성공했으며, 같은 조건의제약MLP및analytic보정과비교해실행가능성과큰장애물의한계를분석했다.

4부 · 한계와 후속

무엇이 아직 미완료인가

완료한 실험과 앞으로 검증할 질문을 구분한다.

증거의 범위

학습 seed는1개다. Sampling seed2개는 서로 다른 학습 결과가 아니다. 개발 source와 이전 장면을 공유하므로 새로운 데이터 분포로의 일반화 주장은 제한된다. 작은 조건도 독립24장면이 아니라12장면×2sampling seed다.

제어와 모델

생성 state의 회전 표현이 정확한 회전행렬 제약을 항상 만족하지 않고, root가 안전해도 신체가 닿을 수 있다. Decoder는 학습 분포를 벗어난 동작을 완벽하게 구현하지 못한다. MLP의 hard bounds도 root 운동학 조건이며 전신 균형을 보장하지 않는다. Direct action head를 예측하면서 실제 실행에는 쓰지 않는 복잡성도 남았다.

계산과 재현

비용 유도는 30Hz 예산을 넘으며 추가 profiling은 장면1개의 짧은 측정이다. 로컬 절대 경로와 외부 MimicKit/Isaac Lab 의존성이 남고, 가중치·원본 모션·학습 데이터는 공개 저장소에 포함하지 않는다. 외부 MimicKit snapshot의 정확한 commit도 아직 식별하지 못했다. Clone 한 번으로 완전 재현된다고 안내하지 않는다.

우선순위가 있는 후속

- Train 상태 여러 개에서 같은 최종goal의 좌우 우회 경험·생성 다양성을 확인한다.
- 필요하면 고정 decoder로 실제 실행 검증한 우회 경험을 양쪽으로 수집한다.
- 두 생성기에 동일 자료·비용을 적용하고 train에서 후보를 고정한다.
- 새 source·장면과 추가 학습seed에서 성공·충돌·정지·latency를 다시 평가한다.

부록 A · 실행 재현

최신 모델과 파일을 찾는 법

파일 이름의 final·best보다 모델 hash와 결과 manifest가 기준이다.

역할	최신 실제 경로
MLP 생성기	outputs/obstacle_repair_v10/h60_models/mlp/model.pt
Diffusion 생성기	outputs/obstacle_repair_v10/h60_models/diffusion/model.pt
공통 decoder	outputs/obstacle_repair_v10/decoder_repair/model.pt
학습 데이터	outputs/obstacle_repair_v10/h60_dataset/data.pt
실행/입력 고정	configs/releases/guidance_comparison_v2_manifest.json
최종 보고서·자료 고정	configs/releases/guidance_comparison_v2_results.json

SHA256 hash는 파일 내용의 식별값이다. 이름이 같아도 가중치 내용이 바뀌면 hash가 달라진다. 실행기는 frozen 목록의 hash를 검사해 입력·코드 변경을 감지한다. 누락된 외부 환경까지 전부 포착하거나 재현성 전체를 보증하는 장치는 아니다.

코드의 역할

핵심 파일	설명
scripts/run_guidance_comparison_v2.py	같은 초기 상태·geometry·목표를 설정하고 물리 평가
planning/feasible_root_optimization.py	MLP bounded root 적분·최적화와 공유 feasibility cost
planning/state_action_sequence.py	local state71·SequenceNet·DDIM·window계약
planning/future_conditioned_action.py	root32특징과공통decoder
scripts/analyze_guidance_comparison_v2.py	범위별집계·실패진단·동일상태검사
scripts/render_guidance_comparison_v2.py	예측/실제실행overlay영상·timeline

위 planning 경로의 공통 접두사는 src/humanoid_imitation/이다. 전체 실행 명령·subprocess exit code·원본 trajectory는 outputs/guidance_comparison_v2에 남았다. 과거 모델과 로그를 삭제하거나 덮어쓰지 않았다.

부록 A · 환경과 명령

기존 로컬 환경에서 다시 평가하기

아래 명령은 필요한 가중치·source·외부 환경이 설치된 경우에만 실행된다.

실험 환경	확인한 버전
GPU	NVIDIA RTX4060Ti8GB
Python / PyTorch	3.11.14 / 2.7.0+cu128
Isaac Lab / Isaac Sim	0.53.1 / 5.1
NVIDIA driver	580.178.04
Backend / 제어	Isaac Lab / position drive, control30Hz·physics120Hz
NumPy / SciPy	1.26.0 / 1.15.3

프로젝트 root에서 실행한다. 예시의 출력 폴더는 아직 없는 새 이름이어야 한다. MLP와DP모두 같은 scene file·source shift·manifest를 사용한다.

```
cd /home/an/Desktop/project_alpha

/home/an/miniconda3/envs/issaclab/bin/python \
  scripts/run_guidance_comparison_v2.py \
  --arm M1 --variant legacy --seed 4504 \
  --manifest configs/releases/guidance_comparison_v2_manifest.json \
  --scene-file reports/assets/guidance_comparison_v2/scenes.json \
  --source-shift 15 --lr 0.05 \
  --out-dir outputs/my_v2_mlp

# 같은 조건의 Diffusion 유도
# 위 명령에서 --arm D2, --guide-scale 15를 사용하고
# --lr을 빼며 out-dir을 새 이름으로 바꾼다.
```

원본 기록을 먼저 본다

README는 빠른 소개, final_project_overview.md는 요약, guidance_comparison_v2.md는 최신 비교 근거다. 이전 closeout의 높은 점수는 곡선 prior가 포함된 다른 조건일 수 있다. 상태가 좋은 checkpoint를 임의로 섞어서 비교하면 공통 실행기 조건이 깨진다.

부록 B · 과거 결과의 해석

이전 점수와 최신 점수를 섞지 않는다

장면·입력·실행 방법이 바뀌면 새로운 실험이다.

실험	결과 예	어떻게 읽나
기존 closeout36장면	MLP출력최적화91.7%, DP유도92.6%	곡선 target이 포함된 비용, DP3samplingseed
Guidance-only V1	route-free DP32/48, 곡선포함46/48	새 geometry24개와이동snapshot, 모델고정
V1 MLP route-free	0/24	예측 속도 폭주가 발견된 부적절 대조
최신 V2	bounded MLP9/24, DP33/48	공유 feasibility cost, train선택후같은 장면재평가
최신 analytic root 기준선	24/24	생성후geometry보정, DP고유능력으로 취급하지않음

기존 closeout은 본 평가·ablation648회에 pilot84·profile6을 더해 738회 물리 실행을 기록했다. V1은264회였다. V2는 새 실행173회와 기존 raw DP48회 재사용이다. 이를 단순히 더해 독립 test 표본 수라고 소개하면 안 된다. 조건이 겹치고 진단·pilot·profile이 섞여 있다.

새 최종 checkpoint가 생겼는가

V1·V2에서는 생성기·decoder 가중치 update가 없었다. 바뀐 것은 비용·추론 최적화·평가 설정이다. “더 학습해서 새로운 모델이 좋아졌다”가 아니라 “고정 모델에 실행 시 처리를 달리해 성능을 비교했다”가 정확하다.

검증과 파일 보존

V2에서는 CPU14검사, 공유 비용의 finite-difference gradient검사(오차약6.1×10^{-11}), 실제초기상태·목표·geometry144개짝비교를통과했다. 이전frozen68개·V2입력/코드73개와이전공개자료hash도확인했다. Hash개수는각manifest범위라서서로중복될수있다.

> **테스트 통과의 의미**
>
> 좌표 변환·차분·큰 입력 제한·비활성 보존 등이 코드대로 작동한다는 증거다. 모든 물리 상황의 안전성이나 성공을 검증한 증거는 아니다.

부록 C · 용어 사전

제어와 학습의 용어

같은 단어가 다른 단계에서 쓰일 때 무엇을 뜻하는지 확인한다.

용어	이 보고서의 의미
Root	몸통의 world 위치·방향을 대표하는 기준
Pose / motion	한 순간의 자세 / 시간에 따른 자세·이동
Action	28DoF에 적용한 관절PD목표
Reference	추종하거나조건으로사용할원하는움직임
Tracker	초기단계의외부reference조건PPO추종정책
Teacher	목표task를수행해경험을모으는PPO+AMP정책
Decoder	미래root특징을실제action으로바꾸는고정지도학습모델
Observation	그순간policy/생성기가받는관측값
State	simulator가가진물리상태;관측과항상같지않음
Rollout	정책을실행하며얻은연속transition기록
Episode	초기화후종료까지의실행단위
Window	학습을위해episode에서자른과거·미래구간
Source group	crop·mirror·branch가공유하는원본계보
Checkpoint	저장된모델가중치와관련설정
Frozen	가중치나입력을고정하고변경하지않는상태
Resume	optimizer·step까지복원한재개;가중치load와구분

핵심은 역할을 이름보다 입출력으로 구분하는 것이다. Teacher와 tracker가 모두 PPO여도 같은 모델이 아니며, 최신 decoder를 PPO tracker라고 부르면 구현과 다른 설명이 된다.

부록 C · 용어 사전

생성·평가의 용어

좋은 숫자가 무엇을 뜻하고 무엇은 뜻하지 않는지 읽는다.

용어	이 보고서의 의미
MLP	context에서미래sequence를한번에예측하는신경망
Diffusion	noise제거를학습해조건부미래분포를모델링
Denoising	noisysequence에서깨끗한sequence를복원
DDIM	적은시점으로복원하는결정론적sampling형태
Guidance	비용gradient를생성과정에적용하는실행시유도
Nominal	비용처리전같은context/noise의원본예측
SDF	장애물경계까지signeddistance
Knot	시간제어신호를보간하기위한대표값,waypoint와다름
Horizon H / execution E	미래예측길이 / 실제재계획간격
Closed loop	실제상태를다시입력해행동을수정하는실행
Prior	학습된동작경향또는고정부모행동모델
Clearance	장애물과계획/신체사이의여유
p50 / p95	측정값의중앙값 / 95백분위
Learning seed	가중치학습의난수조건
Sampling seed	같은가중치에서미래를만드는noise조건
Ablation	특정구성요소를바꾸거나제거해영향을검사
Generalization	학습·개발에없던상황에서도성능이유지되는것

“성공했다”는 말에는 반드시 task·초기상태·문턱·평가횟수가 따라야 한다. 실제 주행이라면 collision·낙상·지연도 함께 고려해야 한다. 생성 모션이 보기 좋다는 것과 실행할 수 있다는 것은 서로 다른 지표다.

부록 D · 참고 연구

어떤 연구를 참고했나

알고리즘의 출처와 이 프로젝트에서 바꾼 부분을 구분한다.

[1] PPO · Schulman et al., 2017

정책의 경험 수집과 clipped policy 업데이트를 교대로 수행하는 RL 기반이다. 본 프로젝트는 MimicKit 구현을 사용했고, 환경·보상·데이터·실험을 구성했다.

Proximal Policy Optimization Algorithms · arxiv.org/abs/1707.06347

[2] DeepMimic · Peng et al., 2018

Reference 모션 모방과 task 목표를 RL로 결합하는 기반 아이디어다. 초기 reference 추종을 설계하는 근거로 사용했다. 논문의 모든 skill·캐릭터·실험을 재현한 것은 아니다.

공식 프로젝트 · xbpeng.github.io/projects/DeepMimic/

[3] AMP · Peng et al., 2021

Reference 같은 움직임에 대한 adversarial style 보상을 통해 task 제어를 학습하는 방법이다. 본 teacher는 여기에 목표·품질·정렬 보완을 결합했다.

Adversarial Motion Priors · arxiv.org/abs/2104.02180

[4] DDIM · Song, Meng, Ermon, ICLR 2021

Denoising 단계를 줄여 생성하는 sampler의 근거다. 현재 코드는 clean x0 예측을 사용하며 16개 시점을 선택한다. 모션의 동역학 타당성을 DDIM 자체가 보장하는 것은 아니다.

Denoising Diffusion Implicit Models · arxiv.org/abs/2010.02502

[5] BeyondMimic · Liao et al., 2025

운동 기술과 guided Diffusion을 연결해 실행 시 새 목적에 적응시키는 연구를 참고했다. 본 프로젝트는 별도의 고정 root-conditioned decoder와 작은 시뮬레이션 범위를 사용한다. 동일 구조·규모·실제 하드웨어 검증을 재현했다고 주장하지 않는다.

From Motion Tracking to Versatile Humanoid Control via Guided Diffusion

부록 D · 근거와 출처

최종 설명의 근거를 추적하는 법

이 문서의 점수는 V2 결과, 구현 설명은 실제 코드와 checkpoint를 기준으로 한다.

설명할 내용	근거 파일
최신 점수·latency·원인 분류	reports/guidance_comparison_v2.md 및 assets/guidance_comparison_v2/summary.json
Episode별 성공·오차·source	reports/assets/guidance_comparison_v2/episodes.csv
사전 조건·train후보·선택	assets/guidance_comparison_v2/protocol.json / selection.json
성공 판정의 정확한 구현	planning/arrival_success.py + V2 runner
MLP·DP학습·dataset규모	outputs/obstacle_repair_v10/h60_models/*/result.json / h60_dataset/result.json
모델크기·구조·입출력	checkpoint CPU검사와state_action_sequence / future_conditioned_action.py
과거 실패·변경 결정	docs/development_history.md / docs/experiments/
외부 코드·모션·AI도움	docs/provenance.md

외부 기반과 개발 도움

PPO·AMP·캐릭터 환경의 기반은 MimicKit이다. 물리 backend는 Isaac Lab/Isaac Sim이다. 사용자가 준비한 Mixamo와 공개 모션의 파생본을 사용했으며 원본 라이선스와 재배포 허용은 별도 확인이 필요해 모션·모델을 Git 추적에서 제외했다.

사용자는 직접 실행 영상을 검토하고 문제와 목표를 제시하며 설계 방향을 결정했다. 구현·디버깅·반복 실행·분석·문서와 PDF 제작에는 AI 도구가 참여했다. 이 보고서는 로그에 없는 수집·실험을 했다고 기록하지 않으며, 검증 근거와 한계를 설명할 수 있는 기준 문서다.

> **최종적으로 남기는 판단**
>
> 작은 장애물에서 비용 유도의 작동 가능성을 검증했고, 같은 조건의 제약 MLP와 비교했으며, 큰 장애물의 실패를 생성 계획과 실제 실행 양쪽에서 진단했다. 다음 연구는 이 한계를 출발점으로 해야 한다. 높은 성공 영상만으로 범위를 넓혀 설명하지 않는다.